



Програмиране за мобилни интернет устройства

УВОД. ЕЗИЦИ ЗА ПРОГРАМИРАНЕ И КОНЦЕПЦИИ В
АНДРОИД. КОТЛИН. ОСНОВНИ КОНЦЕПЦИИ НА
ПРОГРАМНИЯ ЕЗИК

История на Андроид

- ▶ Операционна система предназначена предимно за мобилни устройства, базирана на „линукс ядро“
- ▶ 2005 - Android Inc. закупени от Google. Начало на разработката на Далвийк виртуалната машина;
- ▶ 2007 - Open Handset Alliance (Samsung, Motorola, Sony Ericson, T-Mobile, Toshiba, Vodafone, Google, Intel, Texas Instruments)
- ▶ 2008 - Отваряне на кода на Андроид под лиценз на Apache
 - ▶ T-Mobile G1 телефон – Андроид 1.0, Android Dev Phone 1
 - ▶ липса на виртуална клавиатура (ограничен набор от устройства)
- ▶ 2009 - SDK 1.5 , Андроид 1.6, 2.0



История на Андроид

- ▶ 2008 – 1.0
- ▶ 9 Февруари 2009 - 1.1
- ▶ 30 Април 2009 1.5 Cupcake
- ▶ 15 Септември 2009 1.6 Donut
- ▶ 26 Октомври 2009 2.0/2.1 Eclair
- ▶ 20 Май 2010 2.2 Froyo
- ▶ 6 Декември 2010 2.3 Gingerbread
- ▶ 22 Февруари 2011 3.0 Honeycomb
- ▶ 19 Октомври 2012 4.0 Ice Cream Sandwich
- ▶ 9 Юли 4.1 Jelly Bean
- ▶ 31 Октомври 2013 4.4 KitKat
- ▶ 3 Ноември 2014 5.0 LooliPop
- ▶ 5 Октомври 2015 6.0 Marshmallow
- ▶ 22 Август 2016 7.0 Nougat
- ▶ 21 Август 2017 8.0 Oreo
- ▶ 6 Август 2018 9.0 Pie
- ▶ 3 Септември 2019 10
- ▶ 8 Септември 2020 11
- ▶ 12 Октомври 2021 12
- ▶ 15 Август 2022 13
- ▶ 7 Август 2023 14

РАЗВИТИЕ

- ▶ Gingerbread (2.3.3, 2.3.4)
 - ▶ Bluetooth
 - ▶ Media framework
 - ▶ Speech recognition
 - ▶ подобрение на графични интерфейс
- ▶ Honeycomb (3.0, 3.1, 3.2)
 - ▶ специална оптимизация за устройства с голям екран
 - ▶ 3D десктоп с нови уиджети
 - ▶ Преработен многозадачен механизъм
 - ▶ Нови възможности на уеб браузъра – табове, автоматично попълване на форми, синхронизиране на отметки, private сърфиране и др.
 - ▶ Поддръжка на много-процесорни архитектури.

РАЗВИТИЕ

► Ice Cream Sandwich (4.0)

- Подобрения на графични интерфейс, оптимизация за екрани с висока резолюция
- Подобрен многозадачен режим на работа
- Преоразмеряващи се уиджети
- Нови действия за заключване
- Бърз отговор на повиквания
- Подобрение на въвеждането на текст и проверката на правописа и др.

► Jelly Bean (4.1, 4.2, 4.3)

- Vsync timing
- Triple buffering
- Редуциране на латентността при работа с екрана
- CPU input boost
- Хардуерно ускорение на 2D рендирането
- Многозадачност при многоядрена архитектура
- OpenGL ES 3.0
- Bluetooth Smart технология и др.

Развитие

- ▶ KitKat (4.4)
 - ▶ Оптимизации по управление на паметта
 - ▶ Near field communication (NFC) – транзакции с Host Card Emulation
 - ▶ Фреймуърк за принтиране
 - ▶ Нови механизми на запазване на данни
 - ▶ Сензори с ниска консумация
 - ▶ Анализ на поведението на приложенията и др.

Развитие

- ▶ Lollipop (5.0, 5.1)
 - ▶ Нов подход при изграждане на графичния интерфейс - Material design
 - ▶ Конкурентен достъп до ресурси от екрана
 - ▶ Подобрение на WebView
 - ▶ Снимка на екрана и споделяне
 - ▶ Съобщения при заключен екран
 - ▶ Допълнителни данни за съобщенията
 - ▶ Поддръжка на OpenGL ES 3.1

Lollipop 5.x

- ▶ Подобрение по мултимедийните елементи – камера, аудио
 - ▶ Достъп до мултимедийно съдържание на друго приложение
- ▶ Подобрение по съхранението на данни – избор на директория и гарантиран постоянен достъп до нея
- ▶ Подобрение на мрежовата поддръжка – множество и различни мрежи по едно и също време
- ▶ Bluetooth с ниска консумация
- ▶ Подобрение на NFC
- ▶ Подобрение по отношението на управлението на консумация на енергия – проект Volta

Lollipop 5.x

- ▶ Android Runtime (ART)
 - ▶ Ahead-of-time (AOT) компилация
 - ▶ Подобрен garbage collector
 - ▶ Подобрен debug, инструменти за това
 - ▶ *Ниска толерантност към програмите*
- ▶ Поддръжка на 64-Bit в Android NDK

Marshmallow 6.x

- ▶ Marshmallow(API 23)
 - ▶ Контекстно търсене по ключови думи в рамките на приложения
 - ▶ режим Doze, намаляване скоростта на процесора, докато екранът е изключен
 - ▶ Режим на готовност на приложенията
 - ▶ Работа с пръстови отпечатъци
 - ▶ Директен режим на споделяне
 - ▶ Динамични права за достъп след режим на инсталация
 - ▶ Автоматично архивиране и възобновяване на системата
 - ▶ Адаптивна външна памет
 - ▶ Многопрозоръчен режим
 - ▶ Поддръжка на MIDI интерфейс

Marshmallow 6.x

- ▶ Спиране на поддръжката на Apache HTTP клиент
 - ▶ HttpURLConnection
- ▶ Замяна на OpenSSL с BoringSSL
- ▶ Фактори дизайн шаблон за нотификации
- ▶ Промяна в управлението на достъпа до звук
- ▶ Промяна в организацията на мрежови връзки
- ▶ Стриктна валидация на APK файловете
- ▶ Промяна в политиката на използване на USB порта

Nougat 7.x

- ▶ Nougat (API 24,25)
 - ▶ калибриране на цветовете
 - ▶ Промяна в начина на актуализация на системата (втори системен дял)
 - ▶ VR interface
 - ▶ Нови компилатори на виртуалната машина
 - ▶ Подобрения по интерфейс и различни базови функционалности

Oreo 8.x

- ▶ Oreo (API level 26)
- ▶ Напълно нов подход във всяко ниво на операционната система
- ▶ Подобрения – безброй 😊
 - ▶ Нови изисквания към приложенията
 - ▶ Много методи от предни API са deprecated
 - ▶ Задължителност за използване на новите имплементации
 - ▶ Промяна на организацията на работа на услугите
 - ▶ Използване на *JobIdManager*
 - ▶ Реорганизация на основни процеси в работата на ядрото
 - ▶ Подобряване на сигурността



Pie 9

- ▶ Нови възможности за взаимодействие с потребителите - hybrid gesture/button navigation system
- ▶ Интелигентен отговор навсякъде
- ▶ Интелигентен избор на текст в интерфейса за преглед
- ▶ Избор на изображение в интерфейса за преглед
- ▶ Срезове — части от приложения в търсене
- ▶ По-лесно управление на екранни снимки
- ▶ По-интелигентно управление на батерията
- ▶ По-интелигентен контрол на екрана
- ▶ По-малко досадно завъртане на екрана
- ▶ По-лесен начин за регулиране на шума

10

- ▶ Край на захарната болест
- ▶ Нов интерфейс за Android gestures
- ▶ Нова система за достъп до ресурси



- ▶ Силна еволюция на 10
- ▶ Засилване на сигурността и поверителността
 - ▶ Системата за разрешения
 - ▶ Автоматична отмяна
- ▶ Нов подход за нотификации
- ▶ Ново приложение за потоково видео
- ▶ Native запис на екран
- ▶ Меню на системно ниво за свързани устройства

12,13,14

- ▶ Нов интерфейс
- ▶ Нов интерфейс за съвдаеми телефони и таблети
 - ▶ Chrome подобен интерфейс
 - ▶ Нова система за паралелна обработка
- ▶ Поддръжка на клониране на приложения и едновременна работа на инстанциите му

Андроид устройства

- ▶ Устройства
 - ▶ Смартфони
 - ▶ Таблети
 - ▶ Електронни четци
 - ▶ Netbooks
 - ▶ MP4 плеари
 - ▶ Интернет телевизори
- ▶ голямо разнообразие на устройства - тип/форма, версии на ОС, екрани (резолюция и плътност)

ИЗИСКВАНИЯ

- ▶ ограничени възможности по отношение на консумация на енергия, изчислителна мощ и количество памет
- ▶ необходимост от оптимизиране на всички технологични процеси за да отговорят на изискванията
- ▶ преработване на стандартната JVM – Dalvik VM, два и повече пъти намаляване на размера на файловете (паметта)
- ▶ Java Native Interface (C, C++)

Инструменти

- ▶ JDK;
- ▶ Android SDK;
- ▶ Eclipse IDE;
 - ▶ Android Development Tools (ADT)
- ▶ IntelliJ Idea
- ▶ Android studio
- ▶ Gradle (Gradle plugin)
 - ▶ Зависимости между програмни компоненти
 - ▶ Хранилища за библиотеки
 - ▶ Процес на изграждане на приложения

Android SDK

- ▶ API Android SDK
- ▶ Документация
- ▶ Android Virtual Device – емуляция на андроид устройство
- ▶ Development Tools – инструментални средства за компилиране и внедряване на програмите
- ▶ Sample Code

Версии SDK и Android API Level

- ▶ Android 1.0 (API level 1)
- ▶ Android 1.1 (API level 2)
- ▶ Android 1.5 Cupcake (API level 3)
- ▶ Android 1.6 Donut (API level 4)
- ▶ Android 2.0 Eclair (API level 5)
- ▶ Android 2.0.1 Eclair (API level 6)
- ▶ Android 2.1 Eclair (API level 7)
- ▶ Android 2.2–2.2.3 Froyo (API level 8)
- ▶ Android 2.3–2.3.2 Gingerbread (API level 9)
- ▶ Android 2.3.3–2.3.7 Gingerbread (API level 10)
- ▶ Android 3.0 Honeycomb (API level 11)
- ▶ Android 3.1 Honeycomb (API level 12)
- ▶ Android 3.2 Honeycomb (API level 13)
- ▶ Android 4.0–4.0.2 Ice Cream Sandwich (API level 14)
- ▶ Android 4.0.3–4.0.4 Ice Cream Sandwich (API level 15)
- ▶ Android 4.0.3–4.0.4 Ice Cream Sandwich (API level 15)
- ▶ Android 4.1 Jelly Bean (API level 16)
- ▶ Android 4.2 Jelly Bean (API level 17)
- ▶ Android 4.3 Jelly Bean (API level 18)
- ▶ Android 4.4 KitKat (API level 19)
- ▶ Android Lollipop 5.0-5.1 (API level 20-22)
- ▶ Android Marshmallow 6.0 (API level 23)
- ▶ Android Nougat 7.0 (API level 24-25)
- ▶ Android Oreo 8.0 (API Level 26)
- ▶ Android 8.1 (API level 27)
- ▶ Android 9 (API level 28)
- ▶ Android 10 (API level 29)
- ▶ Android 11 (API level 30)
- ▶ Android 12 (API level 31,32)
- ▶ Android 13 (API Level 33)
- ▶ Android 14 (API Level 34)

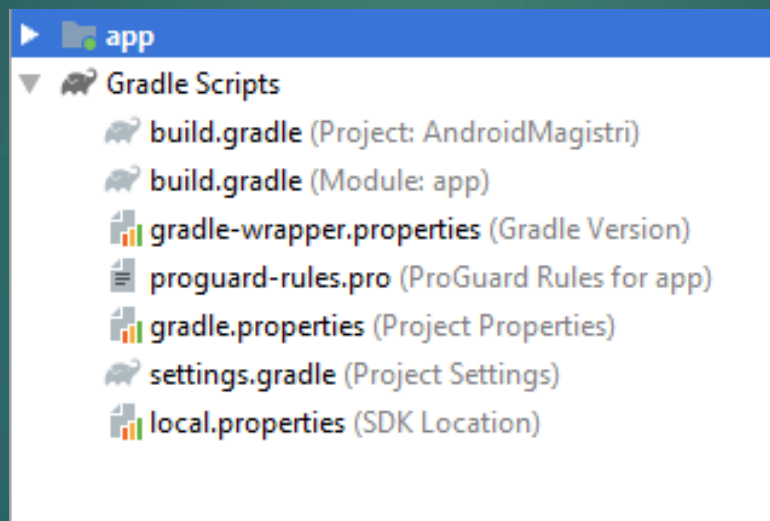
Инструменти

- ▶ Езици за програмиране
 - ▶ Java
 - ▶ Kotlin
 - ▶ 2011
 - ▶ Project Kotlin
 - ▶ 2016 – обявен като независим език за програмиране

Инструмент за създаване и управление на Android проект

- ▶ Gradle
 - ▶ Самостоятелна система за създаване компилиране и управление
 - ▶ Андроид студио - плъгин
 - ▶ JVM инструмент
 - ▶ Java, Groovy
 - ▶ Scala
 - ▶ Компиляция при промяна
 - ▶ Многокомпонентна поддръжка
 - ▶ Йерархия от скриптове
 - ▶ Нива по модули

Компилиране на проект и изпълнение. Gradle.



Въведение в Котлин

- ▶ JVM език
 - ▶ Свой компилатор и среда за изпълнение
 - ▶ По-лесен синтаксис
 - ▶ По-бързо изпълнение
 - ▶ По-добра поддръжка на функции и ламбда изрази
 - ▶ По-добра поддръжка на корутини и асинхронно програмиране
- ▶ Функционална Java

Променливи и типове данни

- ▶ Kotlin поддържа различни типове променливи:
- ▶ `var`: Използва се за променливи, които могат да бъдат променяни (mutable).
- ▶ `val`: Използва се за константи (immutable).
- ▶ Пример:
- ▶ `var age: Int = 30`
- ▶ `val name: String = "Ivan"`

ПРОМЕНЛИВИ И ТИПОВЕ ДАНИИ

- ▶ Типове данни в Kotlin:
 - ▶ Цели числа: Byte, Short, Int, Long
 - ▶ Дробни числа: Float, Double
 - ▶ Логически: Boolean
 - ▶ Текстови: Char, String
- ▶ Пример:
 - ▶ `val isActive: Boolean = true`
 - ▶ `val temperature: Float = 36.6f`

Условни оператори

- ▶ Условните оператори в Kotlin включват:
 - ▶ if, else if, else: Използват се за изпълнение на различни блокове от код в зависимост от условията.

- ▶ Пример:

```
val score = 85
if (score >= 90) {
    println("Отличен")
} else if (score >= 75) {
    println("Много добър")
} else {
    println("Добър")
}
```

Условни оператори

- ▶ when: - Подобно на switch в други езици

- ▶ Пример:

```
val grade = 'A'
when (grade) {
    'A' -> println("Отличен")
    'B' -> println("Много добър")
    'C' -> println("Добър")
    else -> println("Неуспешен")
}
```

ЦИКЛИ

- ▶ Kotlin поддържа няколко вида цикли:
 - ▶ `for`: Използва се за итериране през колекции и диапазони.
- ▶ Пример:

```
for (i in 1..5) {  
    println(i)  
}
```

ЦИКЛИ

- ▶ `while`: Изпълнява блок от код, докато условието е вярно.
- ▶ `do-while`: Изпълнява блок от код поне веднъж, след което продължава, докато условието е вярно.

- ▶ Пример:

```
var i = 5
while (i > 0) {
    println(i)
    i--
}
```


ЦИКЛИ

- ▶ Пример за do-while:

```
var j = 5
```

```
do {
```

```
    println(j)
```

```
    j--
```

```
} while (j > 0)
```

ФУНКЦИИ

- ▶ Функциите са основни градивни блокове в Kotlin
 - ▶ **Глобални функции:** Функции, които са дефинирани директно в пакет, без да са част от клас или обект. Те могат да бъдат достъпвани от всяка точка на кода чрез директно извикване или чрез импорт.
 - ▶ **Член-функции (методи):** Функции, които са дефинирани вътре в класове или обекти и са свързани с техните инстанции.
 - ▶ **Разширителни функции:** Специален вид функции, които позволяват добавяне на нови методи към съществуващи класове, без да променят техния код.

Глобална функция

// Файл: Utils.kt

```
package com.example.utils
```

```
fun sum(a: Int, b: Int): Int {  
    return a + b  
}
```

// Достъп до функцията от друг файл в същия проект

```
import com.example.utils.sum
```

```
fun main() {  
    val result = sum(3, 5)  
    println(result) // Output: 8  
}
```

Член функция (метод)

```
class Calculator {  
    fun multiply(a: Int, b: Int): Int {  
        return a * b  
    }  
}  
  
fun main() {  
    val calculator = Calculator()  
    val product = calculator.multiply(3, 5)  
    println(product) // Output: 15  
}
```

Разширителна функция

```
fun String.removeFirstLastChar(): String {  
    return this.substring(1, this.length - 1)  
}
```

```
fun main() {  
    val str = "Hello"  
    println(str.removeFirstLastChar()) // Output: ell  
}
```

ФУНКЦИИ

- ▶ Функции с връщане на стойност:

- ▶ Пример:

```
fun add(a: Int, b: Int): Int {  
    return a + b  
}
```

- ▶ Викане на функция и използване на върната стойност:

- ▶ `val sum = add(3, 5)`
- ▶ `println("Sum: \">$sum")`

Класове и Обекти

- ▶ Основи на класове и обекти в Kotlin:
- ▶ Пример за клас:
 - ▶ `class Person(val name: String, var age: Int)`
- ▶ Създаване на обект:
 - ▶ `val person = Person("Ivan", 30)`
 - ▶ `println(person.name)`

Класове и Обекти

- ▶ Методи в класове и обекти:

- ▶ Пример за метод:

```
class Person(val name: String, var age: Int) {  
    fun greet() {  
        println("Hello, my name is \"$name\"")  
    }  
}
```

- ▶ Използване на метод:

- ▶ `val person = Person("Ivan", 30)`
- ▶ `person.greet()`

Интерфейси и Абстрактни класове

- ▶ Kotlin поддържа интерфейси и абстрактни класове:

- ▶ Интерфейси:

```
interface Drivable {  
    fun drive()  
}
```

- ▶ Абстрактни класове:

```
abstract class Vehicle {  
    abstract fun start()  
    fun stop() {  
        println("Vehicle stopped")  
    }  
}
```

Интерфейси и Абстрактни класове

► Пример:

```
class Car: Vehicle(), Drivable {  
    override fun start() {  
        println("Car started")  
    }  
  
    override fun drive() {  
        println("Car is driving")  
    }  
}
```

```
val myCar = Car()  
myCar.start()  
myCar.drive()  
myCar.stop()
```

Обработка на ИЗКЛЮЧЕНИЯ

- ▶ Kotlin поддържа обработка на изключения чрез try-catch блокове:
- ▶ Пример:

```
fun divide(a: Int, b: Int): Int {  
    return try {  
        a / b  
    } catch (e: ArithmeticException) {  
        println("Деление на нула")  
        0  
    }  
}
```

- ▶ `val result = divide(10, 0)`
- ▶ `println("Result: \${result}")`

Ламбда функции и ВИСОКО НИВО НА ФУНКЦИИ

- ▶ Ламбда функции:

- ▶ Пример:

```
val sum = { a: Int, b: Int -> a + b }  
println("Sum: \${sum(3, 4)}")
```

- ▶ Високо ниво на функции:

```
fun operate(x: Int, y: Int, operation: (Int, Int) -> Int): Int {  
    return operation(x, y)  
}
```

- ▶ Пример:

```
val result = operate(5, 3, sum)  
println("Result: \${result}")
```



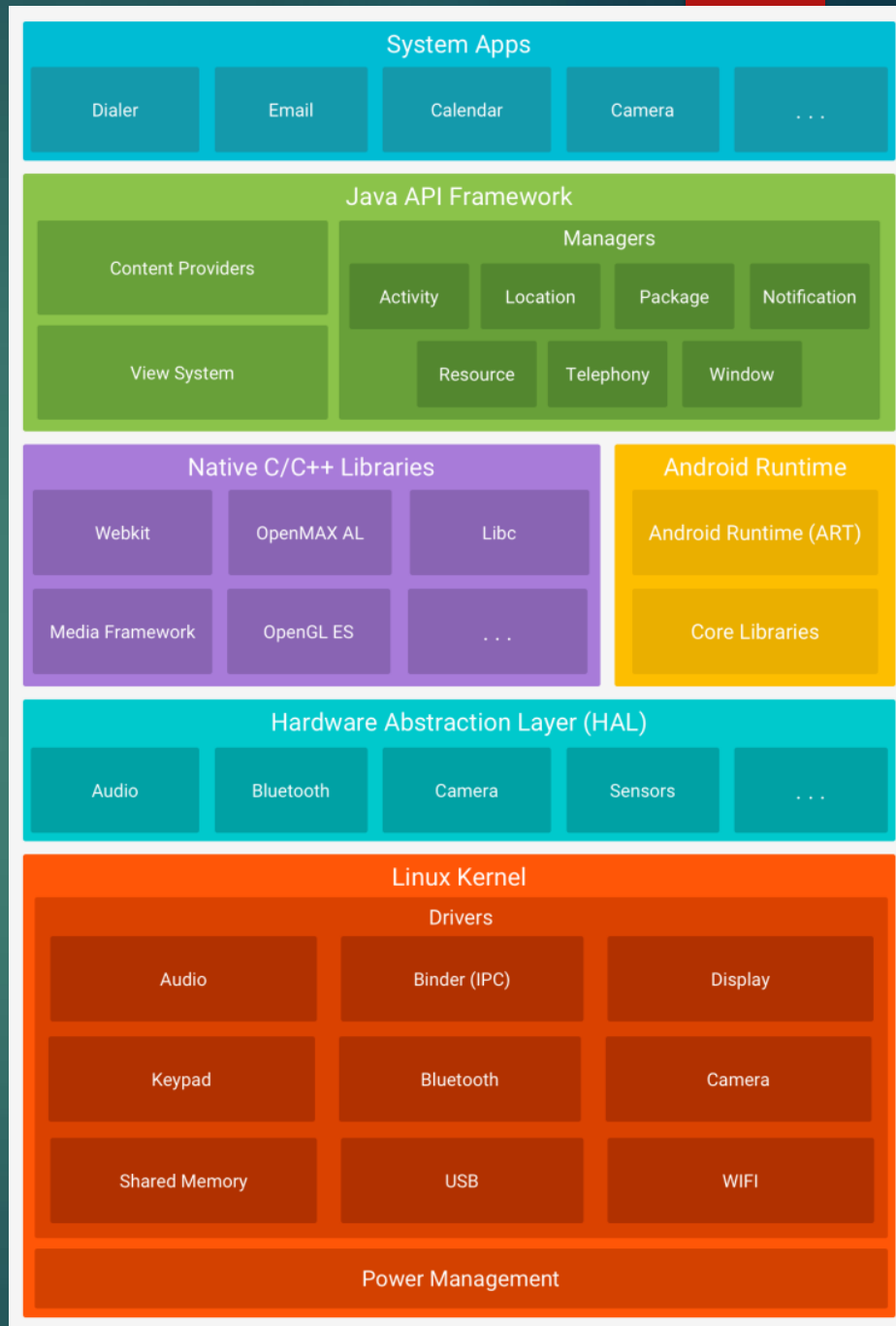
Програмиране за мобилни интернет устройства

КОНЦЕПЦИИ НА АНДРОИД ОС

Архитектура

Структура.

- Ядро (Linux kernel);
- HAL
- Библиотеки, run-time среда, Далвийк виртуална машина, ART;
- Андроид фреймуърк;
- Андроид приложения (базови, потребителски).



Фиг.1 Архитектура на
Андроид ОС

Ядро

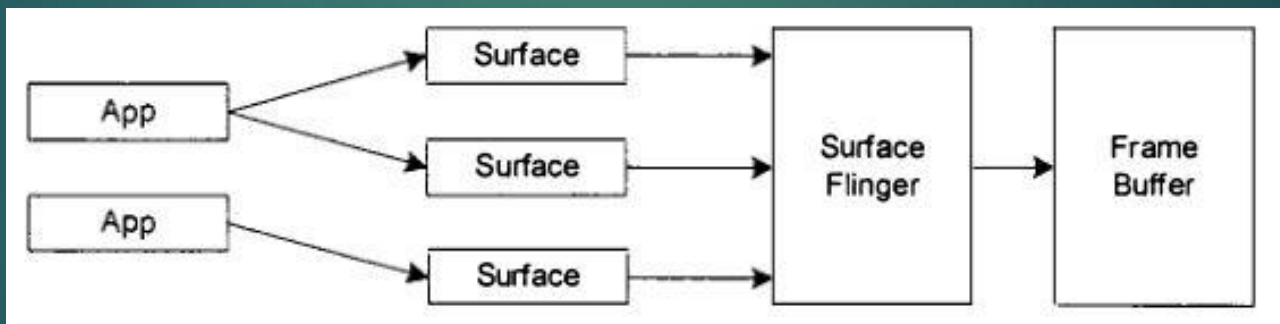
- ▶ Абстракция между хардуера и останалата част от програмния стек;
 - ▶ Базирано на Linux kernel 3.4, 3.10 (4.x), 3.16 (5.x) , 3.18 (6.x), 4.4.1 (7.x) , 4.10 (8.x).... 5.15.0
 - ▶ Управление на паметта
 - ▶ Междупроцесна комуникация
 - ▶ Сигурност
 - ▶ Файлови и мрежови операции
 - ▶ Управление на драйверите
 - ▶ Специфични за Андроид допълнения и разширения
 - ▶ управление на захранването;
 - ▶ специфична за Андроид междупроцесна комуникация
 - ▶ управление на общата памет (Shared memory)
 - ▶ действия при малко свободна памет

Слой на хардуерна абстракция

- ▶ Интерфейс за производителите – съвкупност от интерфейси
 - ▶ Модули и устройства

Библиотечен слой

- ▶ С системна библиотека (libc) – Bionic (2.3)
 - ▶ разработка на Google, многократно зареждане - 200kB.
 - ▶ оптимизирана за максимално бързодействие
- ▶ Контролер на екрана
 - ▶ масиви от битове за изображенията



Фиг.2 Многослойна обработка на базата на двоен буфер

Библиотечен слой

- ▶ Функционални библиотеки

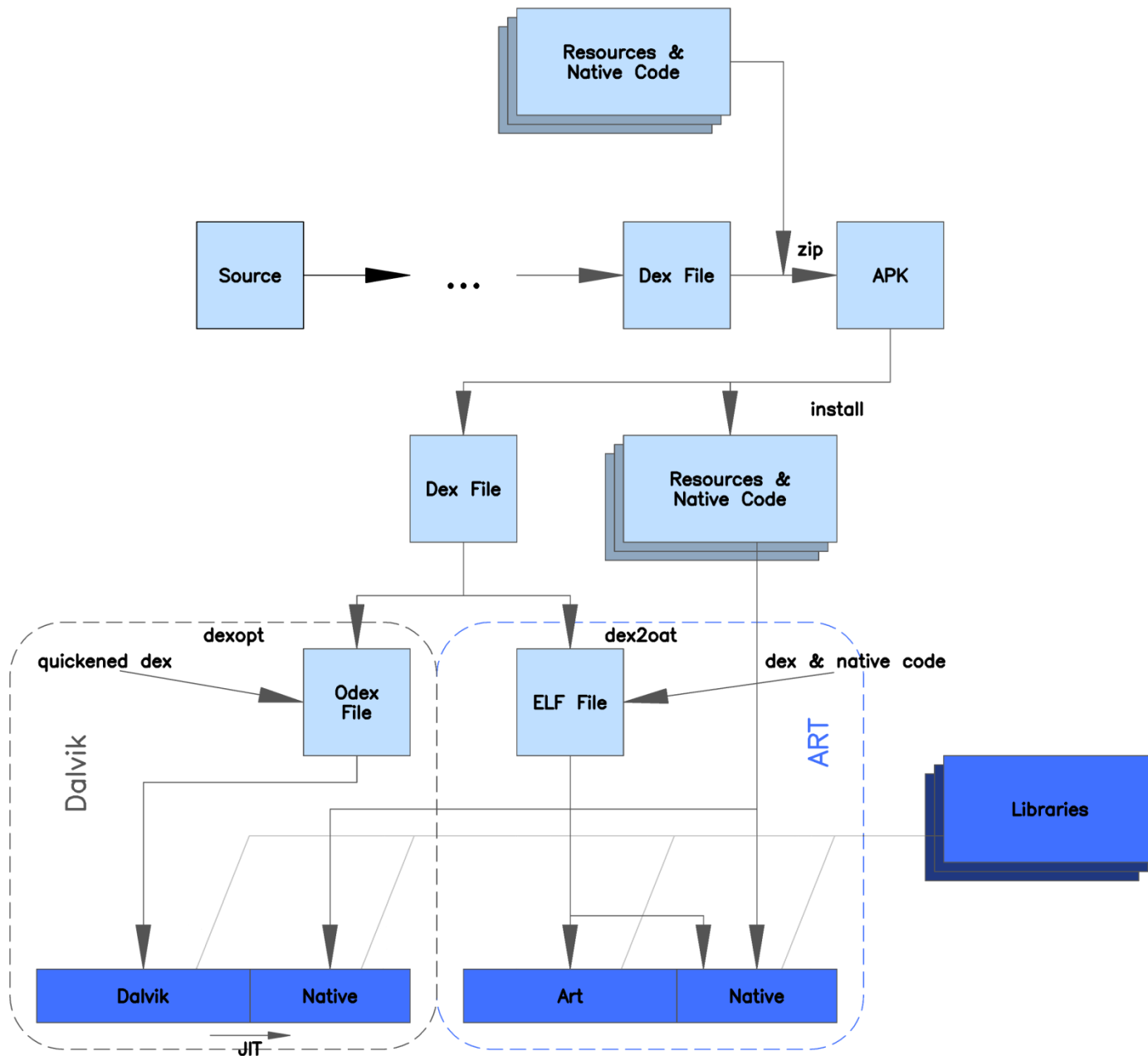
- ▶ медиа фреймуорк;
- ▶ SQLite;
- ▶ OpenGL ES;
- ▶ FreeType;
- ▶ WebKit;
- ▶ SGL;
- ▶ SSL.

Среда за изпълнение

- ▶ Виртуална машина
 - ▶ Далвийк виртуална машина– регистрова организация, оптимизирана за Risk процесори, абстракция от хардуера;
 - ▶ бавни процесори
 - ▶ малко RAM
 - ▶ лимитиран живот на батерията
- ▶ *.dex .odex файлове
- ▶ базови Java библиотеки

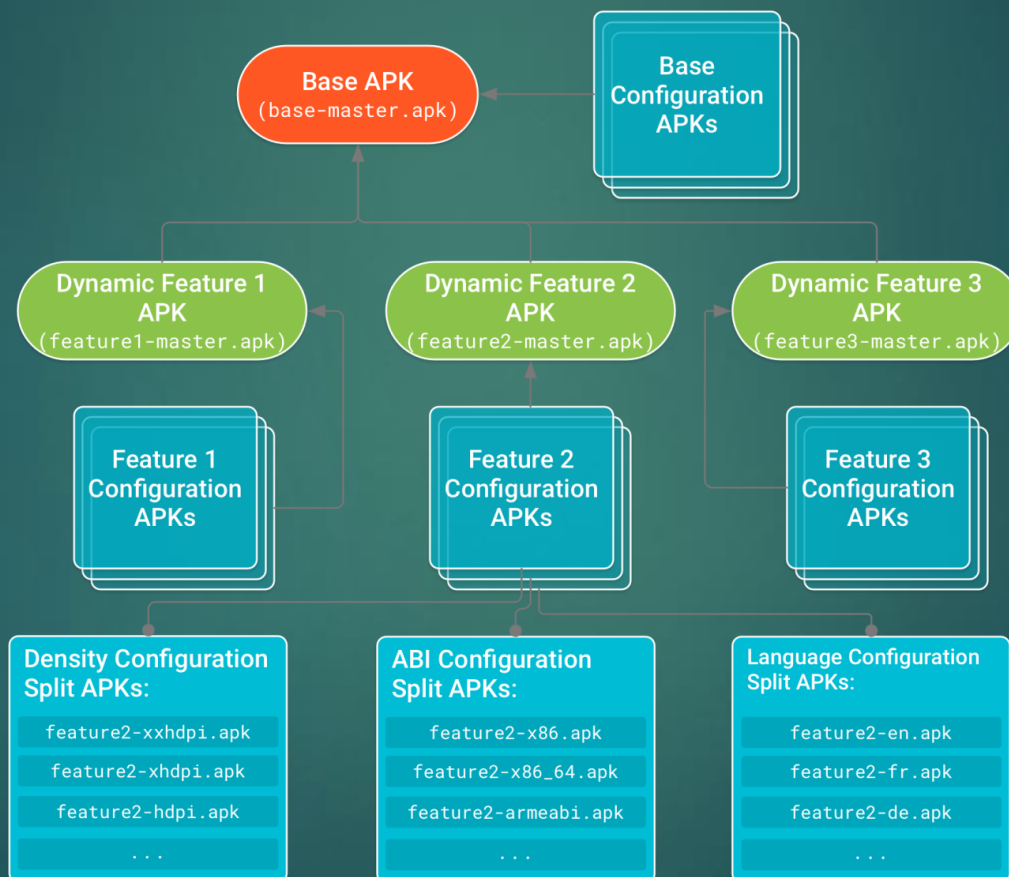
Android runtime

- ▶ Android runtime (ART)
 - ▶ ahead-of-time (AOT) компилиране
 - ▶ Dex2oat инструмент
 - ▶ .elf файлове
- ▶ подобро почистване на ненужни данни
- ▶ подобрения в областта на развоя и откриването на грешки



Split .apk

► Концепция за сепариране



Фреймуърк на приложенията

- ▶ управление на жизнения цикъл на приложенията, управление на ресурси и др.
 - ▶ Managers
 - ▶ Activity Manager
 - ▶ Package Manager
 - ▶ Window Manager
 - ▶ Resource Manager
 - ▶ и др.
 - ▶ Content Providers
 - ▶ View System

Приложен слой

- ▶ SMS
- ▶ контакти
- ▶ браузър
- ▶ календар
- ▶ навигационни карти и др.

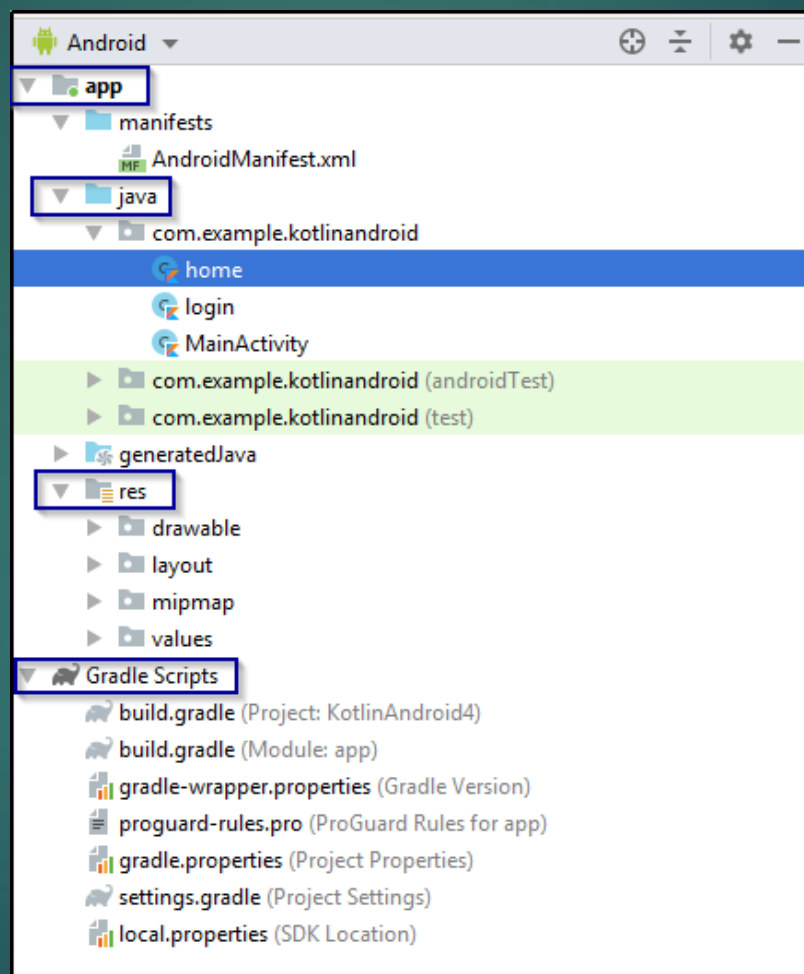
Същност на Андроид приложенията

- ▶ .apk
- ▶ Многопотребителска операционна Линукс базирана система:
 - ▶ Всяко приложение е различен потребител
 - ▶ user ID
 - ▶ Собствена виртуална машина
 - ▶ Индивидуален Линукс процес със собствен жизнен цикъл
- ▶ Принцип на най-ниската привилегия
- ▶ Споделяне на ресурси
 - ▶ Едно и също user ID
 - ▶ Достъп до данни – предоставяне на достъп по време на инсталацията, динамично в новите версии след 5.0

Компоненти на приложенията

- ▶ activity – единичен екран с потребителски интерфейс
- ▶ service – продължителен процес във фонов режим
- ▶ content provider – услуга за управление на ресурси
- ▶ broadcast receiver – бродкаст сигнал в системата
- ▶ intent – механизъм за активиране на компоненти

Структура на проект



Описател на приложението

- ▶ AndroidManifest.xml
- ▶ Дефиниране на минимална версия на програмен интерфейс
- ▶ Дефиниране на права за достъп необходими на приложението
- ▶ Деклариране на хардуерни изисквания
- ▶ Деклариране на допълнителни библиотеки
- ▶ Деклариране на компоненти
 - ▶ <activity> , <service> , <receiver> , <provider>

AndroidManifest

```
<?xml version="1.0" encoding="utf-8"?>

<manifest xmlns:android="http://schemas.android.com/apk/res/android"

    package="com.example.myapplication"

    android:versionCode="1"

    android:versionName="1.0">

    <uses-permission android:name="android.permission.INTERNET" />

    <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />

    <application

        android:icon="@mipmap/ic_launcher"

        android:label="@string/app_name"

        android:theme="@style/AppTheme">

        <activity

            android:name=".MainActivity"

            android:exported="true">

            <intent-filter>

                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />

            </intent-filter>

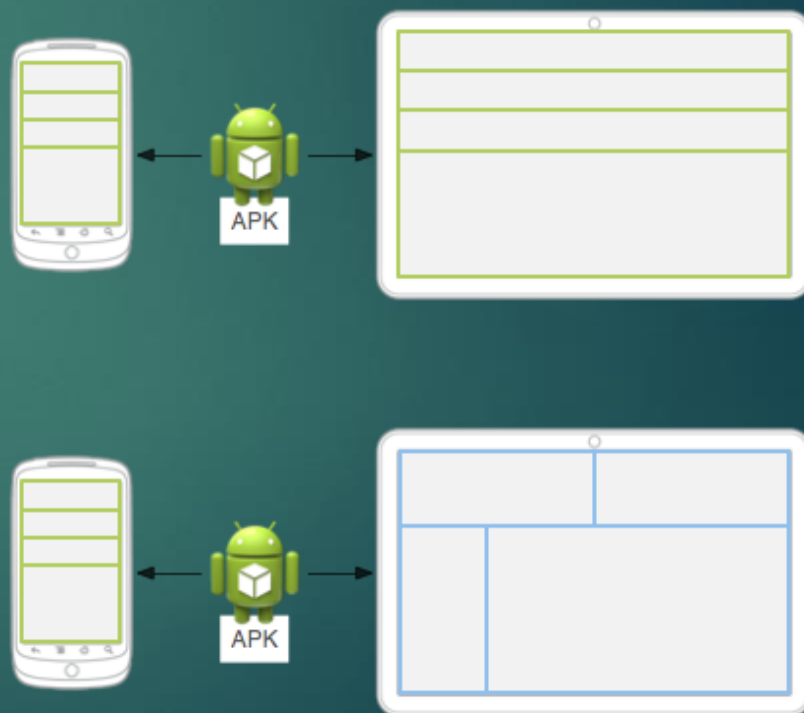
        </activity>

    </application>

</manifest>
```

Ресурси в Android приложения

- ▶ Принцип на ресурсите
- ▶ По подразбиране, алтернативен
 - ▶ `res/layout/`, `res/layout-land/`



Управление на ресурси

- ▶ Предоставяне на ресурси
- ▶ Достъпване на ресурси
- ▶ Прихващане на промени в реално време
- ▶ Локализации
- ▶ Типове ресурси

Предоставяне на ресурси

- animator/ - XML файлове с анимация по характеристики
 - anim/ - XML файлове със стандартна анимация
 - color/ - цветове на състоянието
 - drawable/ - .png, .9.png, .jpg, .gif) или XML компилиран до обект тип изображение
 - mipmap/ - варианти на икони за стартиране на приложения
 - layout/ - графични шаблони
 - menu/ - файлове за дефиниране на менюта
 - raw/ - сурови данни
 - values/ - декларации на ресурси String, Integer, Colors
 - xml/ - стандартни XML файлове
-
- Никога в res/ !!!

```
MyProject/  
  src/  
    MainActivity.java  
  res/  
    drawable/  
      graphic.png  
    layout/  
      main.xml  
      info.xml  
    mipmap/  
      icon.png  
    values/  
      strings.xml
```


Алтернативни ресурси

```
res/  
  drawable/  
    icon.png  
    background.png  
  drawable-hdpi/  
    icon.png  
    background.png
```

- drawable-hdpi-port/
- drawable-port-hdpi/

Достъпване на ресурси

- ▶ Според типа на ресурса
 - ▶ string, drawable, layout...
- ▶ Според името на ресурса
 - ▶ Името на обекта или чрез *android:name*
- ▶ *R.string.hello* - *@string/hello*
- ▶

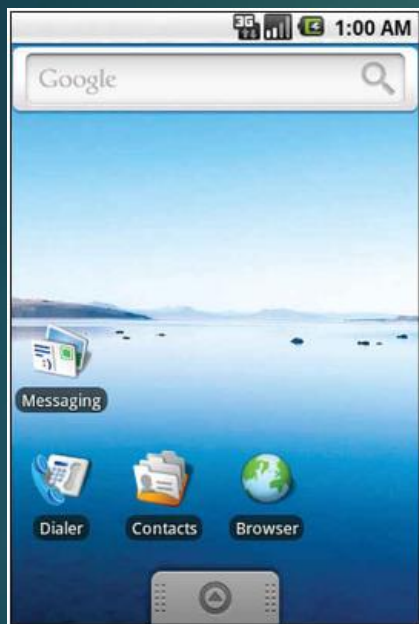
```
val imageView = findViewById<ImageView>(R.id.imageView)  
imageView.setImageResource(R.drawable.myimage);
```



Програмиране за мобилни интернет устройства

ОРГАНИЗАЦИЯ НА ПОТРЕБИТЕЛСКИЯ ИНТЕРФЕЙС (ПИ).
ИЗГРАЖДАНЕ НА ПИ В XML И JAVA. ОСНОВНИ
КОМПОНЕНТИ НА ПОТРЕБИТЕЛСКИЯ ИНТЕРФЕЙС

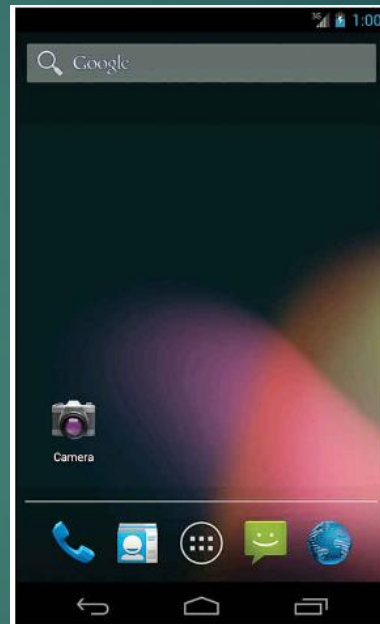
Развитие на графичния интерфейс на Android



1.6



2.3

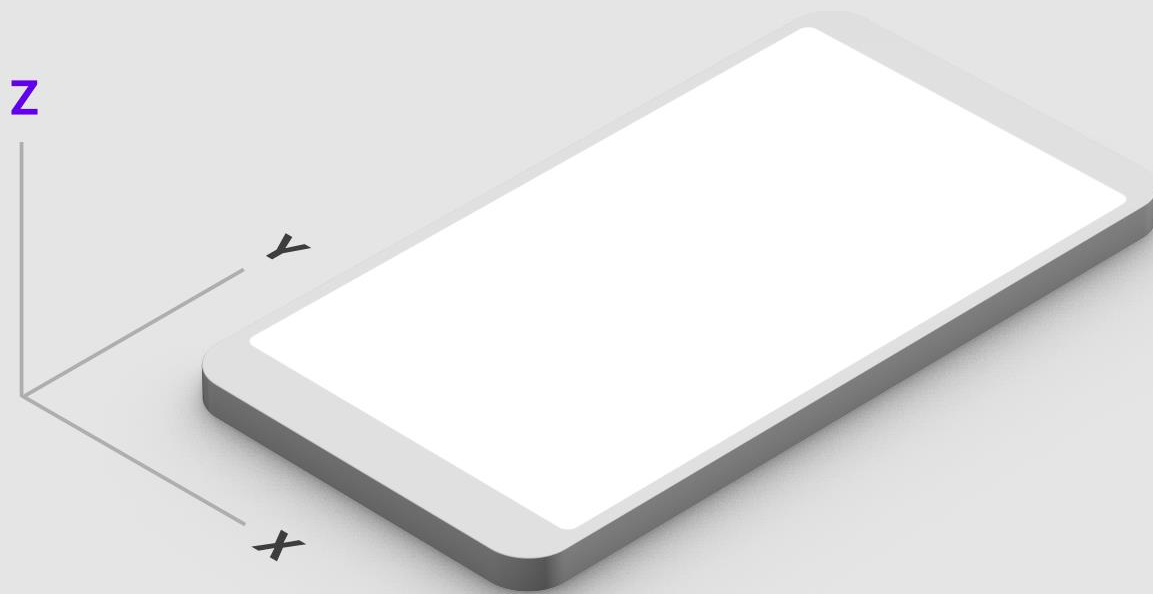


4.2

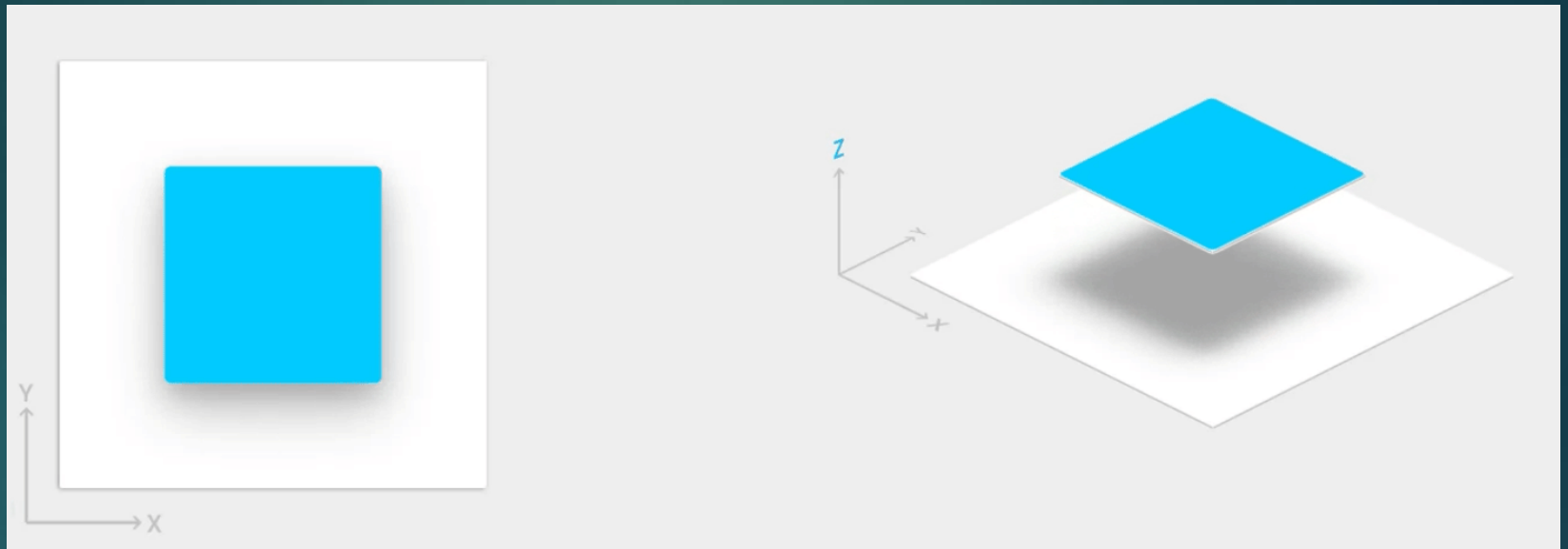


5.0

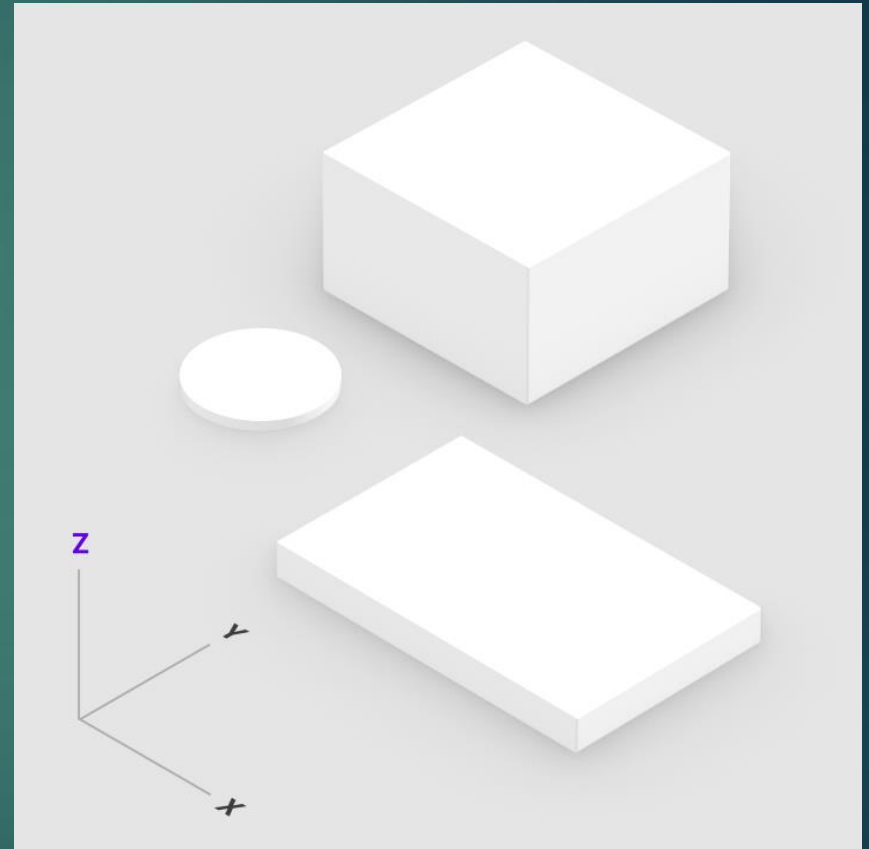
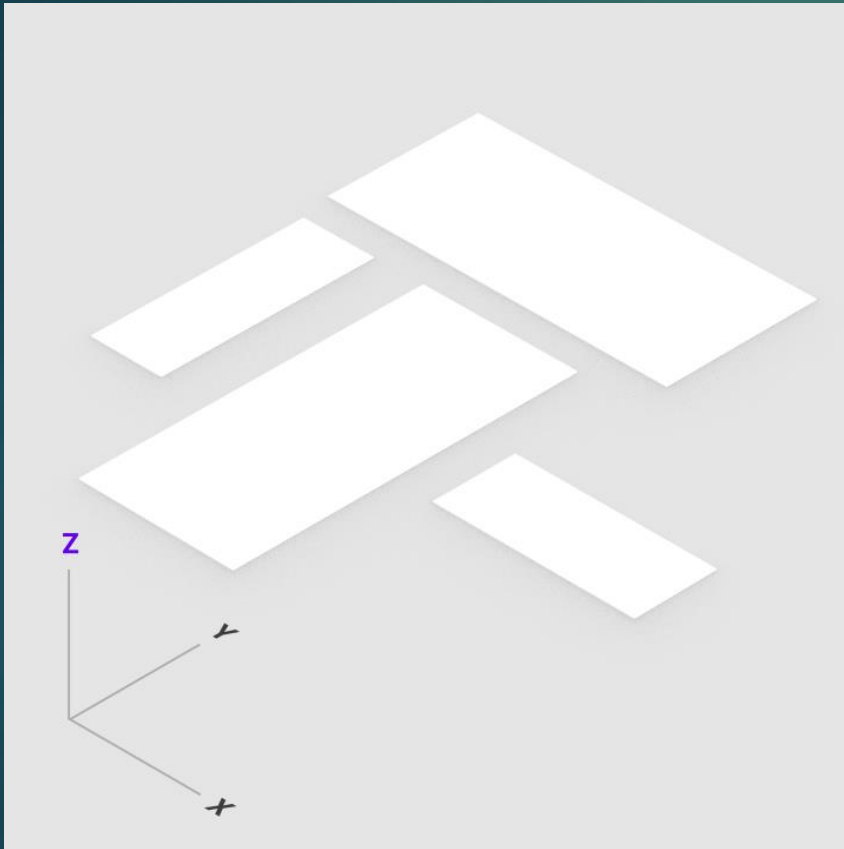
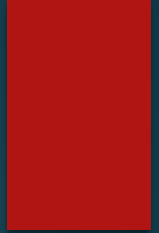
Material Design



Material Design

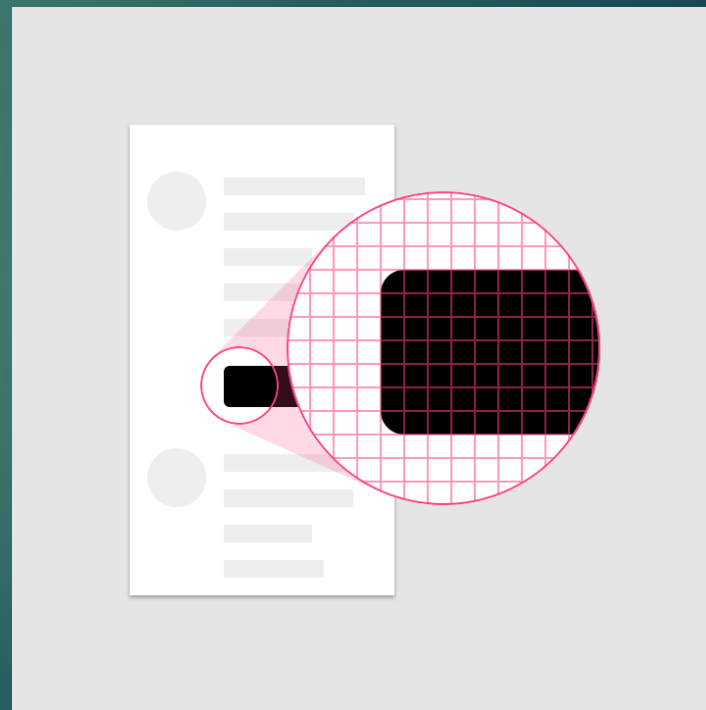
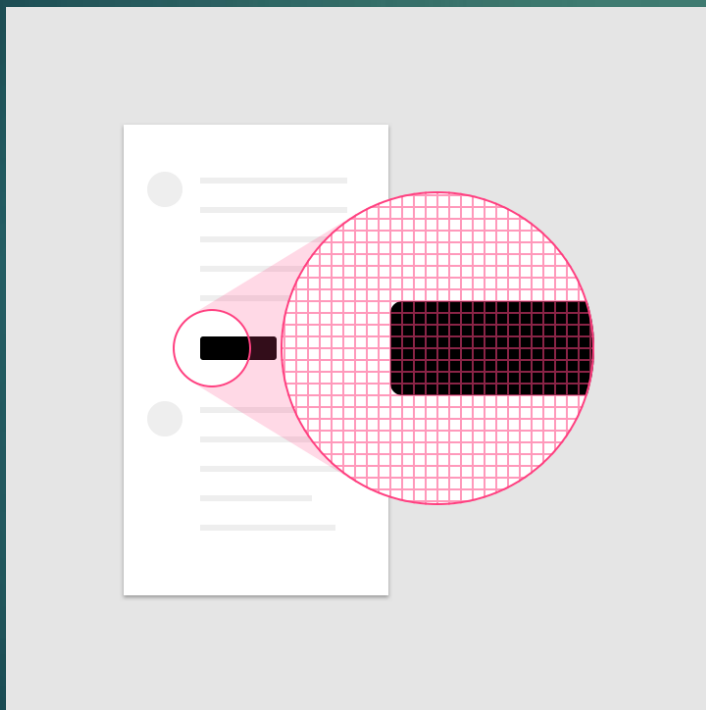


Material Design



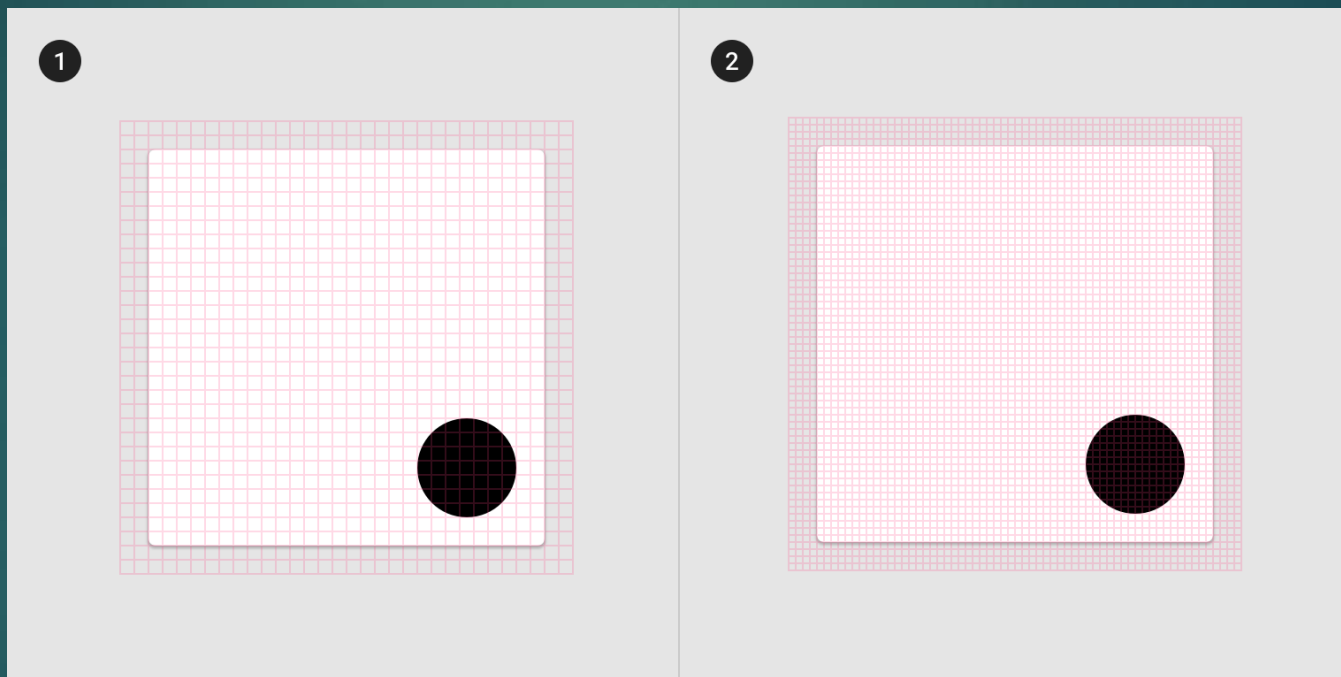
Характеристики на екраните

- ▶ Плътност на пикселите (Pixel density) - Броят пиксели, които се вписват в инч



АДАПТИВНОСТ НА ИЗГЛЕДА

- *density-independent pixels* (dp)
- *scalable pixels* (sp)



АДАПТИВНОСТ НА ИЗГЛЕДА

- ▶ *density-independent pixels* (dp) - абстрактна мерна единица, която се базира на физическата плътност на екрана. 1 dp е еквивалентен на 1 пиксел на 160 dpi (dots per inch) екран, който се счита за базовата плътност.
 - ▶ Основно се използва за измерване на дължини и ширини на елементи на интерфейса (като View компоненти) и разстояния (като margin и padding).
 - ▶ Автоматично се адаптира спрямо различните плътности на екрана, което осигурява еднакъв физически размер на елемента на различни устройства.

АДАПТИВНОСТ НА ИЗГЛЕДА

- **dp** се използва за всички UI елементи с изключение на текстове.
- **sp** се използва основно за текстови елементи.
- **dp** се мащабира само спрямо плътността на екрана.
- **sp** се мащабира както спрямо плътността на екрана, така и спрямо предпочитанията на потребителя за размер на текста.

АДАПТИВНОСТ НА ИЗГЛЕДА

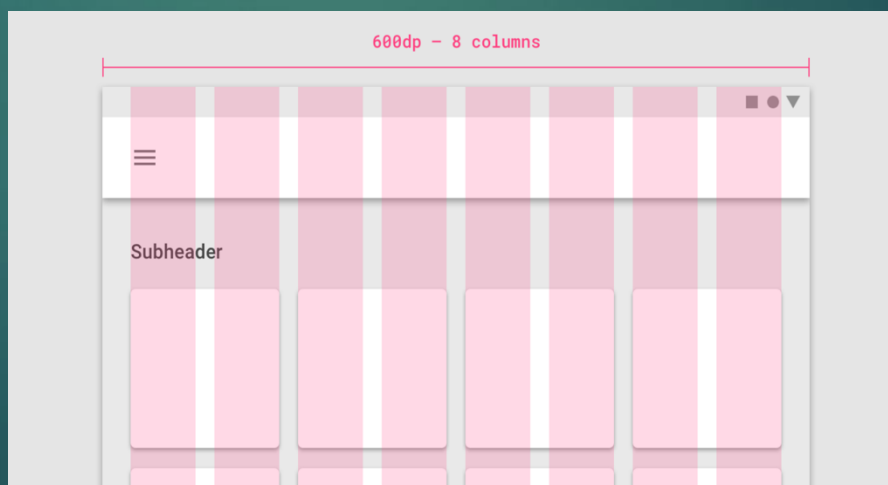
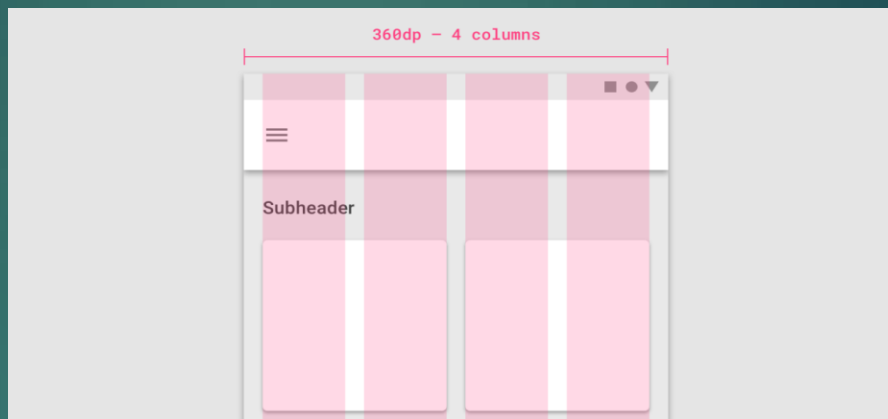
- ▶ *scalable pixels* (sp) - sp е също като dp, но освен да отчита плътността на екрана, тя също така отчита и потребителските настройки за размер на шрифта (font scaling).
 - ▶ Използва се основно за размерите на текстове, за да осигури, че текстът е четлив и съобразен с предпочитанията на потребителя за големина на шрифта.
 - ▶ sp не само се адаптира към плътността на екрана, но също така се мащабира спрямо потребителските настройки за размер на текста, което може да се настрои в настройките на устройството.

Характеристики на екрани

Abbreviation	Name (Dots per Inch)	Density	Scale
LDPI	Low	120	0.75
MDPI	Medium	160	1
HDPI	High	240	1.5
XHDPI	Extra high	320	2
XXHDPI	Extra, extra high	480	3
XXXHDPI	Extra, extra, extra high	640	4

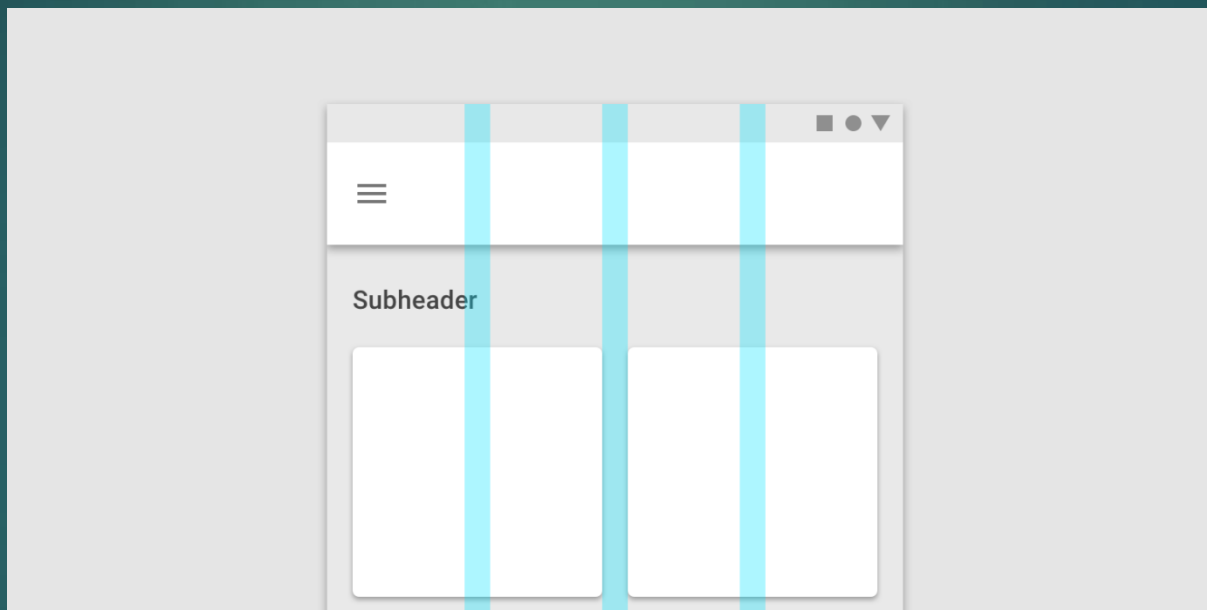
Техники на разполагане на елементите

► Колони



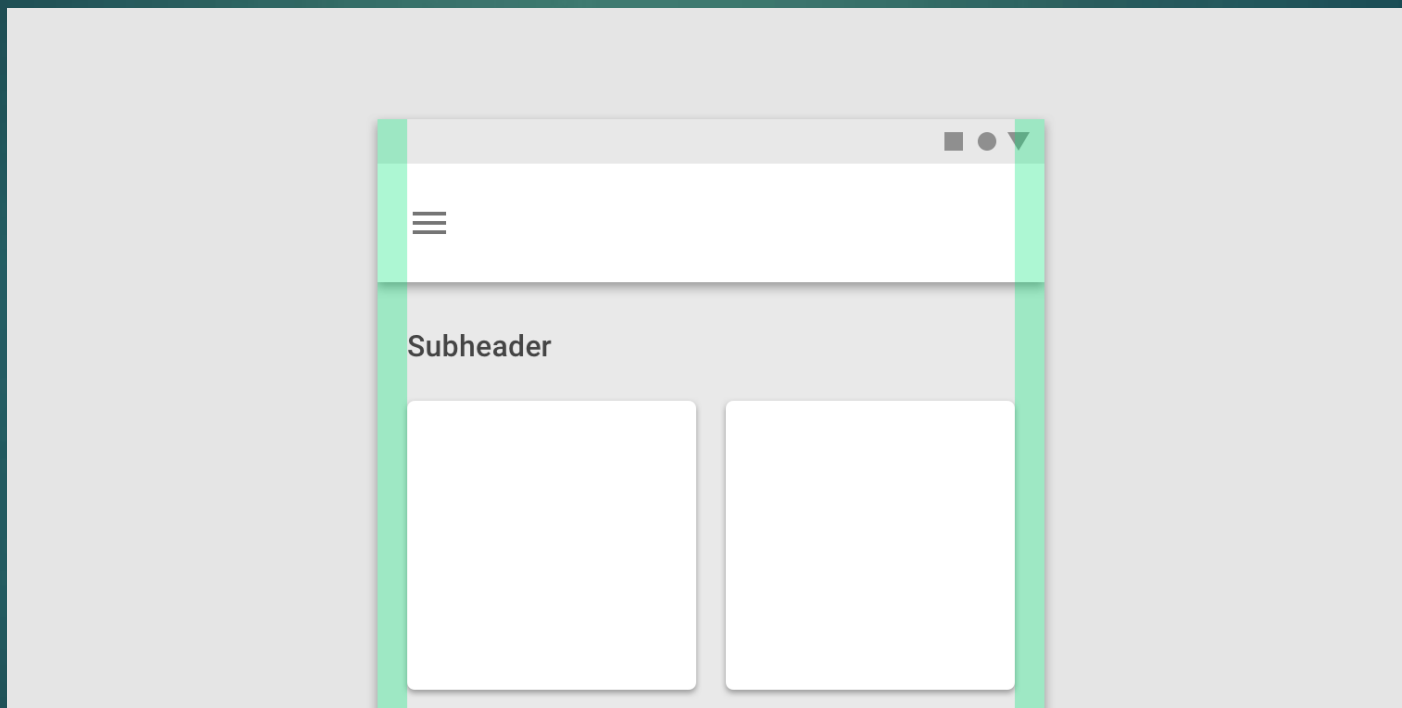
Техники на разполагане на елементите

- ▶ Отстояния между колоните (Gutters)



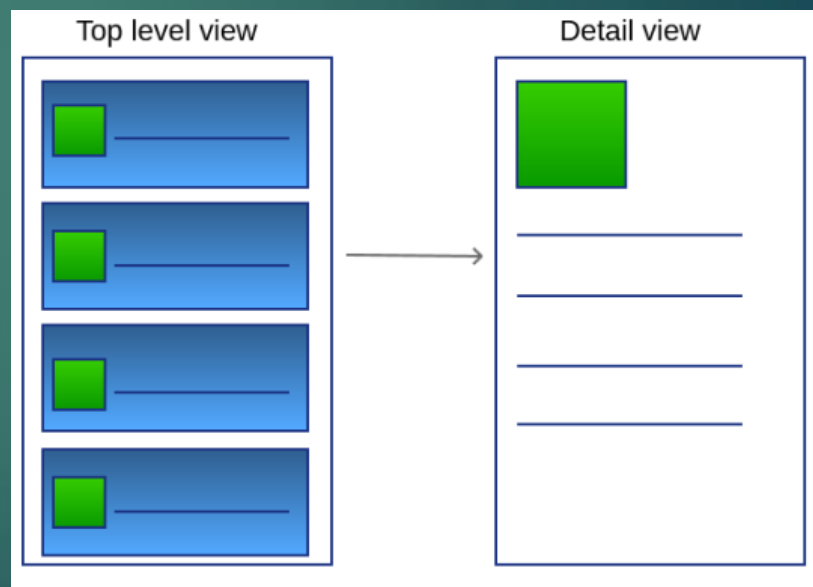
Техники на разполагане на елементите

- ▶ Отстояния между съдържанието и края на екрана (Margins)



Концепции за UI в Android

- ▶ top-level view
 - ▶ Табове, спинъри, Navigation drawer
- ▶ detail/level view



Action Bar развитие

- ▶ Android 3.0 (API level 11)
 - ▶ Иконы
 - ▶ Заглавия
 - ▶ Менюта
- ▶ Toolbar (Android 5.0 Lollipop)
- ▶ TopAppBar (Jetpack Compose)

Стандартни компоненти

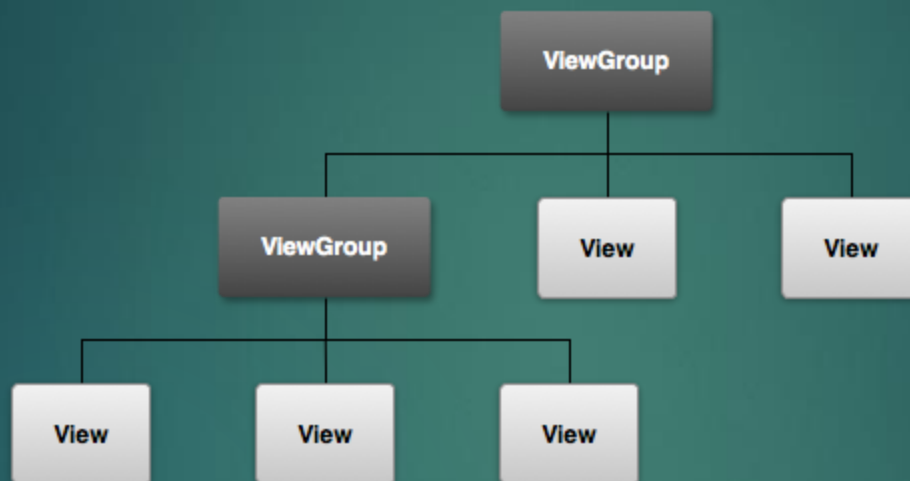
- ▶ View
- ▶ Viewgroup
- ▶ Activity
- ▶ Fragment

Стандартни UI компоненти

- ▶ Tabs
- ▶ Spinners
- ▶ Pickers
- ▶ Lists
- ▶ Buttons
- ▶ Dialogs
- ▶ Grid lists
- ▶ TextFields

Организация

- ▶ Изгледи , група от изгледи



Принципи на изграждане на потребителския интерфейс

► XML базиран подход

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="horizontal">
```

```
<TextView
    android:id="@+id/textView1"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Name"/>
```

```
<EditText
    android:id="@+id/editText1"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="" />
```

```
</LinearLayout>
```

Атрибути

- ▶ ID

- ▶ `<TextView`
`android:id="@+id/textView1"`

....

- ▶ `TextView textView = findViewById(R.id.textView1);`

Атрибути

- ▶ Позиции – правоъгълник с :
 - ▶ Координатни точки - left , top (X,Y)
 - ▶ Размерности width, height
- ▶ Намиране на относителна спрямо родителя позиция
 - ▶ `getLeft()` , `getTop()`
 - ▶ `getRight()`, `getBottom()`
 - ▶ `getLeft() + getWidth()`

Прехващане на събитие

- ▶ В XML изглед
 - ▶ `android:onClick="onClick"/>`
 - ▶ **public void** `onClick(View view)`
- ▶ В Java контролер - listener
- ▶

```
.setOnClickListener(new View.OnClickListener() {  
    @Override  
    public void onClick(View view) {  
        // обработка на събитието  
    }  
});
```
- ▶ Интерфейси за слушатели:
 - ▶ `OnClickListener`, `OnLongClickListener`, `OnTouchListener`,
`OnDragListener`, `OnFocusChangeListener`, `OnHoverListener`

КОНТРОЛИ ЗА ВХОДА

- ▶ Бутони, чекбоксове, радиобутони, текстови полета и др.

- ▶ Бутон

- ▶ Текст с класа Button

- ▶ `<Button`
 `android:layout_width="wrap_content"`
 `android:layout_height="wrap_content"`
 `android:text="@string/button_text"`
 `... />`

- ▶ С иконка и класа ImageButton

- ▶ `<ImageButton`
 `android:layout_width="wrap_content"`
 `android:layout_height="wrap_content"`
 `android:src="@drawable/button_icon"`
 `... />`

- ▶ Текст с иконка и класа Button

- ▶ `<Button`
 `android:layout_width="wrap_content"`
 `android:layout_height="wrap_content"`
 `android:text="@string/button_text"`
 `android:drawableLeft="@drawable/button_icon"`
 `... />`

Текстови полета

- ▶ `<EditText>`
- ▶ Различна роля – номер, дата, парола имейл адрес
 - ▶ `android:inputType="textEmailAddress" />`
 - ▶ `"text,,", "textEmailAddress,,", "textUri,,", "number,,", "phone,,",`
- ▶ Поведения
 - ▶ `"textCapSentences,,", "textCapWords,,", "textAutoCorrect,,", "textPassword,,", "textMultiLine,,",`
- ▶ `<EditText`
 - `android:id="@+id/postal_address"`
 - `android:layout_width="fill_parent"`
 - `android:layout_height="wrap_content"`
 - `android:hint="@string/postal_address_hint"`
 - `android:inputType="textPostalAddress| textCapWords| textNoSuggestions" />`

Чекбокс

► CheckBox

```
► <?xml version="1.0" encoding="utf-8"?>
  <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent">
    <CheckBox android:id="@+id/checkbox_meat"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="@string/meat"
      android:onClick="onCheckboxClicked"/>
    <CheckBox android:id="@+id/checkbox_cheese"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="@string/cheese"
      android:onClick="onCheckboxClicked"/>
  </LinearLayout>
```

Чекбокс

```
▶ public void onCheckboxClicked(View view) {  
    boolean checked = ((CheckBox) view).isChecked();  
    switch(view.getId()) {  
        case R.id.checkbox_meat:  
            if (checked)  
                // обработка  
            else  
                // обработка  
            break;  
        case R.id.checkbox_cheese:  
            if (checked)  
                // обработка  
            else  
                // обработка  
            break;  
        // обработка  
    }  
}
```

Радио бутони

- ▶ RadioButton

- ▶ `<RadioGroup xmlns:android=„http://schemas.android.com/apk/res/android“`
- ▶ `<RadioButton android:id="@+id/....`
- ▶ RadioGroup наследник на LinearLayout с вертикална ориентация по подразбиране

Действие

```
▶ public void onRadioButtonClicked(View view) {  
    boolean checked = ((RadioButton) view).isChecked();  
    switch(view.getId()) {  
        case R.id.radio_pirates:  
            if (checked)  
                // обработка  
            break;  
        case R.id.radio_ninjas:  
            if (checked)  
                // обработка  
            break;  
    }  
}
```

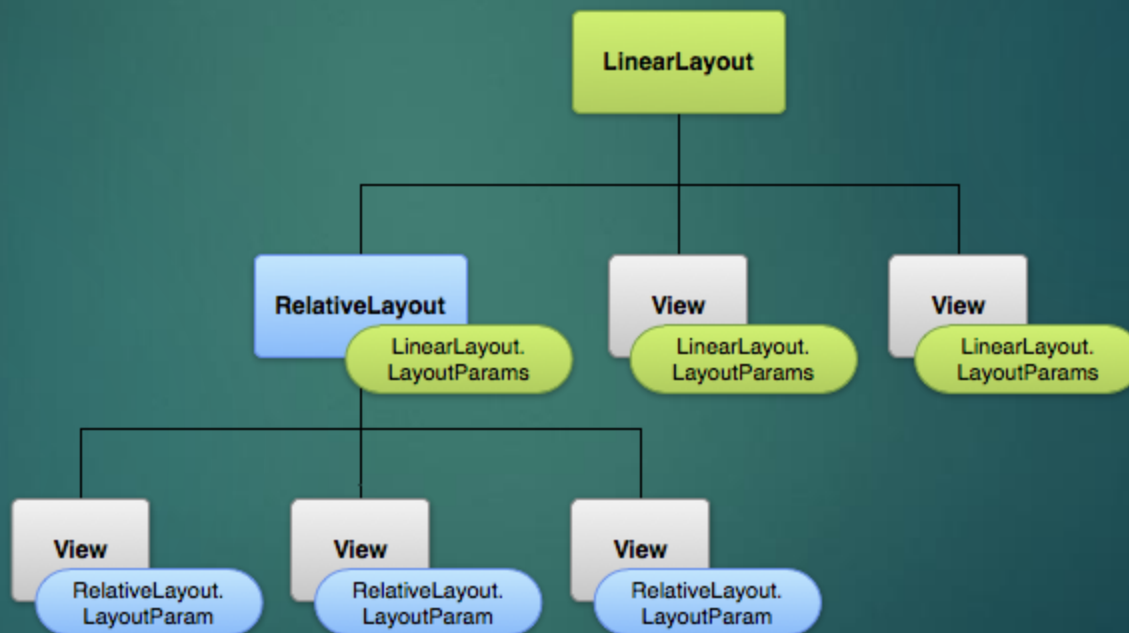
Схеми (Layouts)

- ▶ Деклариране на елемент от потребителския графичен интерфейс в XML
- ▶ Инстанциране на елемент по време на изпълнението

Атрибути

- ▶ Параметри на схемата
- ▶ *wrap_content* - задава размер подходящ за съдържанието
- ▶ *match_parent* (*fill_parent* преди API Level 8) – големина както на родителския

изглед



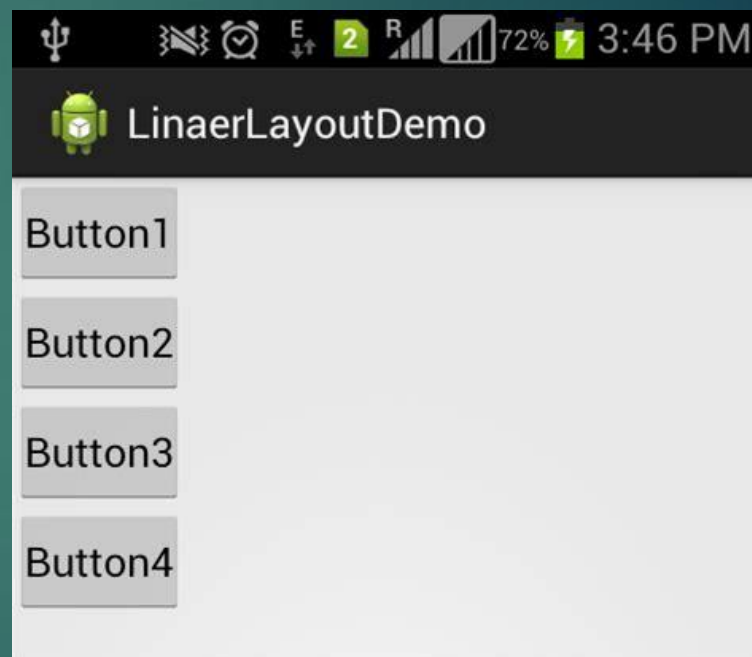
Зареждане на схема

- ▶ Всяка схема е ресурс

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_linear_layout_demo);
}
```

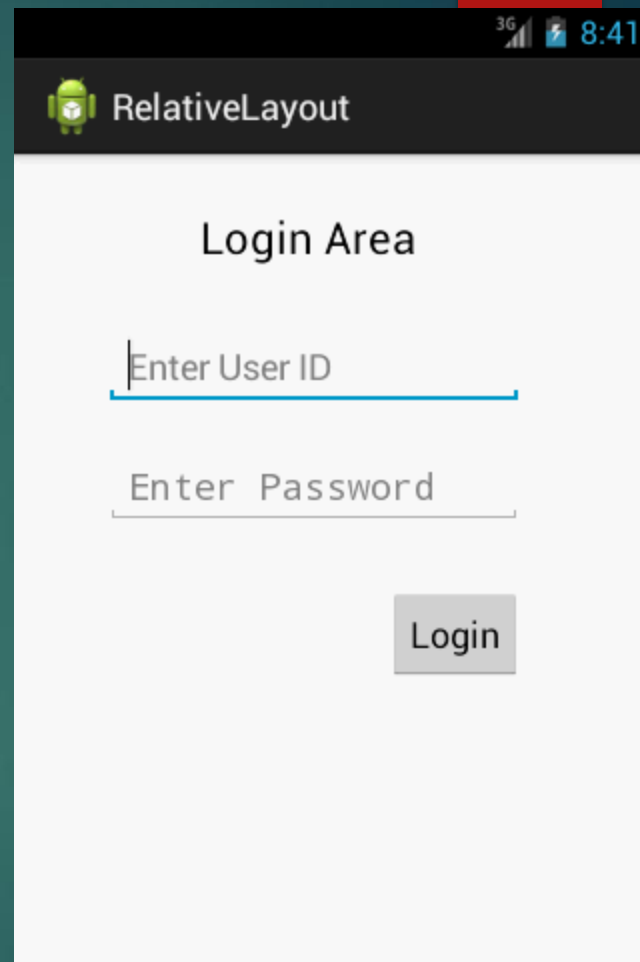
ОСНОВНИ СХЕМИ

- ▶ Linear Layout
 - ▶ android:orientation
 - ▶ android:layout_weight



ОСНОВНИ СХЕМИ

- ▶ Relative Layout – принцип на атрибута true/false
 - ▶ android:layout_alignParentTop
 - ▶ android:layout_alignParentBottom
 - ▶ android:layout_centerVertical
 - ▶ android:layout_below
 - ▶ android:layout_toRightOf



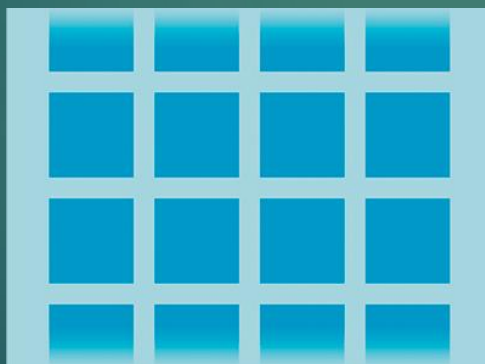
ОСНОВНИ СХЕМИ

- ▶ При изпълнение на операция взаимодействие с адаптер

- ▶ ListView

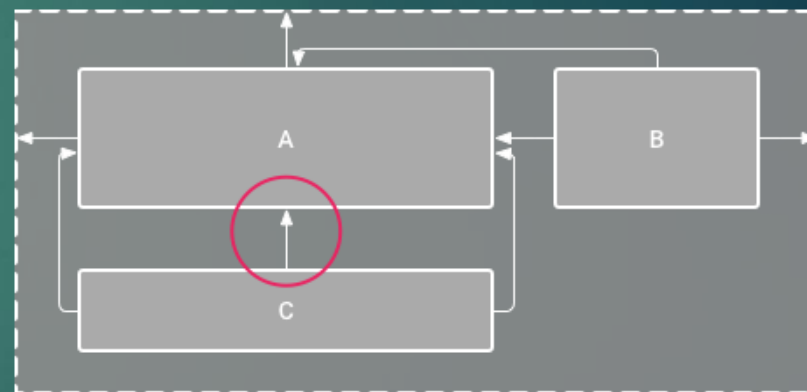
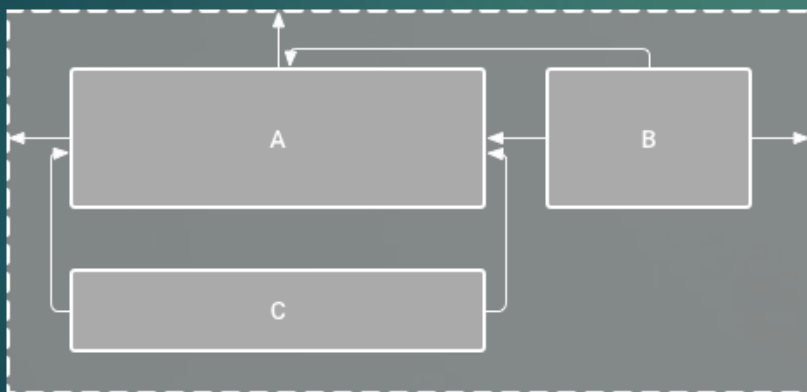


- ▶ Grid View



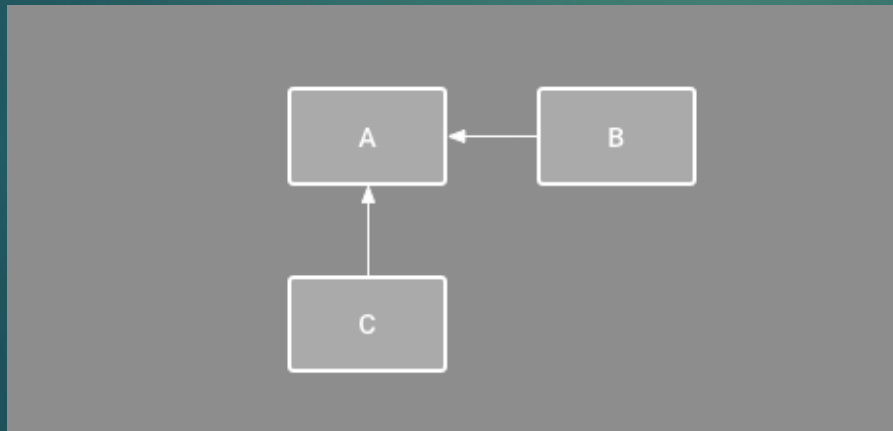
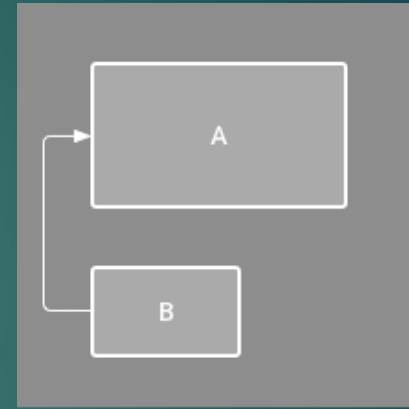
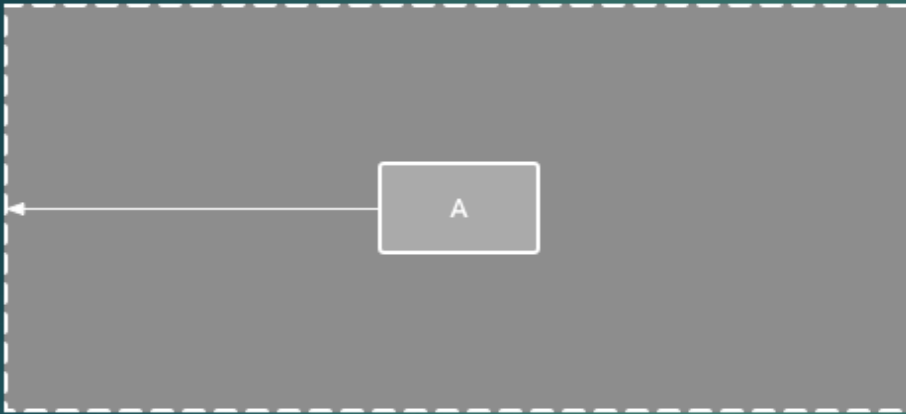
ConstraintLayout

- ▶ Концепция за релативност но с плосък модел на структурата
- ▶ Удобен за създаване и редактиране чрез редактора на схеми
- ▶ Необходими минимум две ограничения

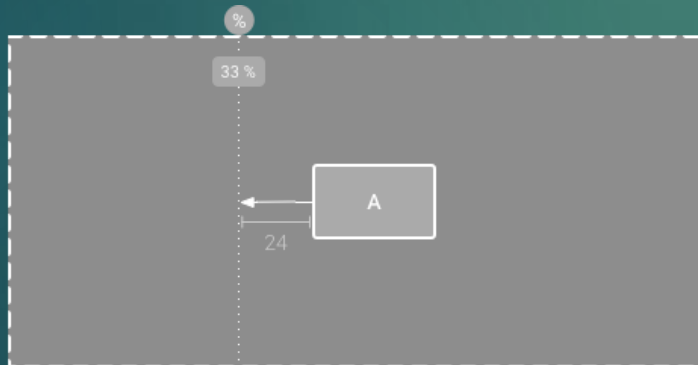
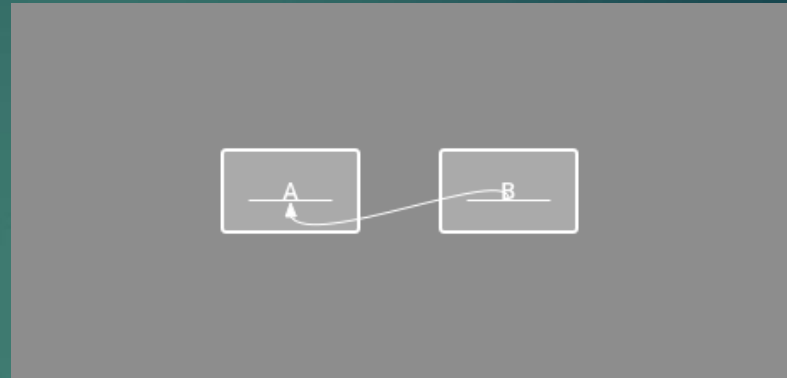
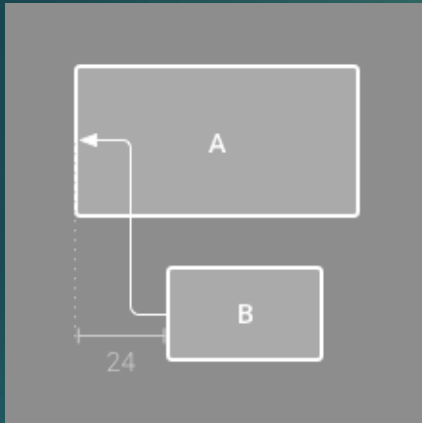


- ▶ Infer Constraints

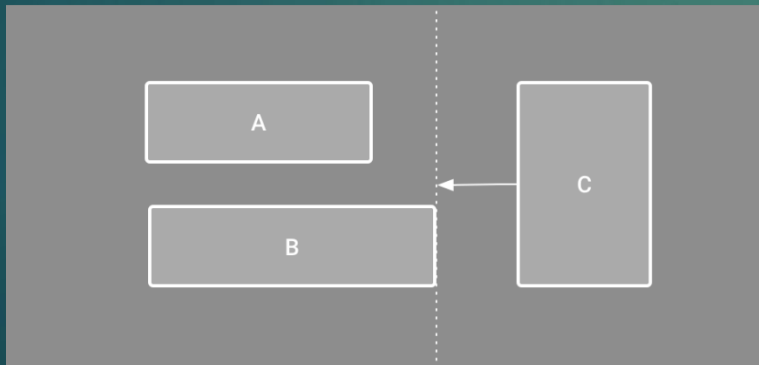
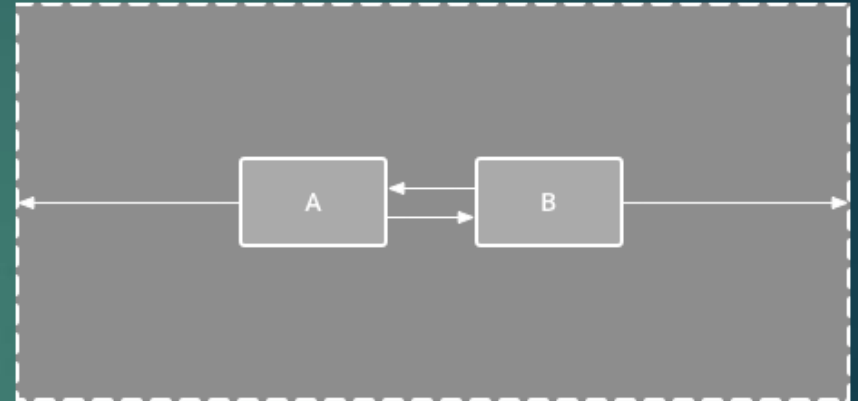
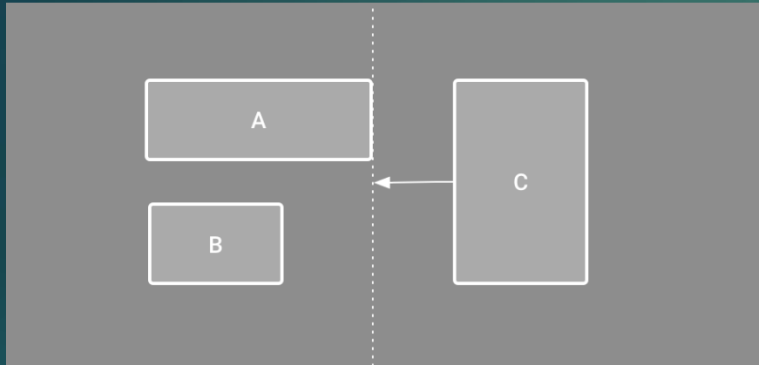
ConstraintLayout



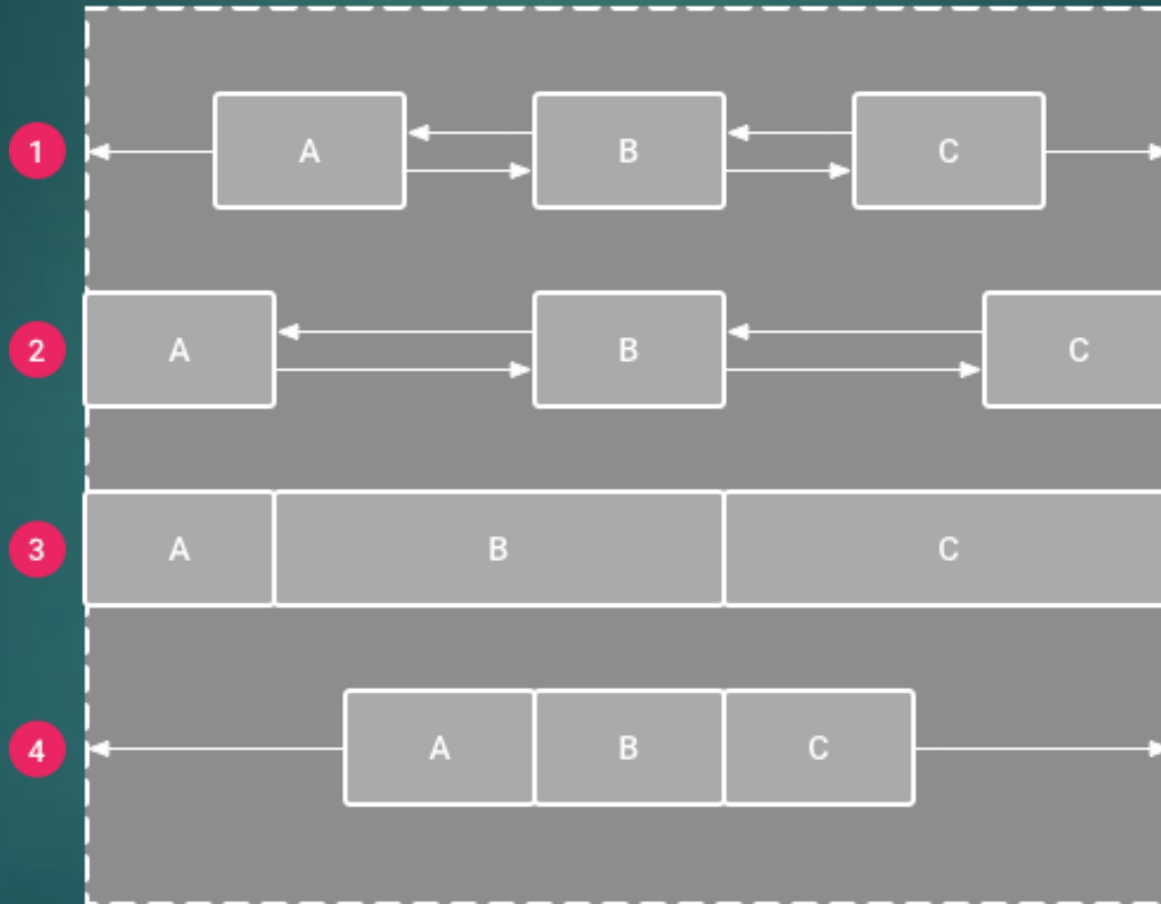
ConstraintLayout



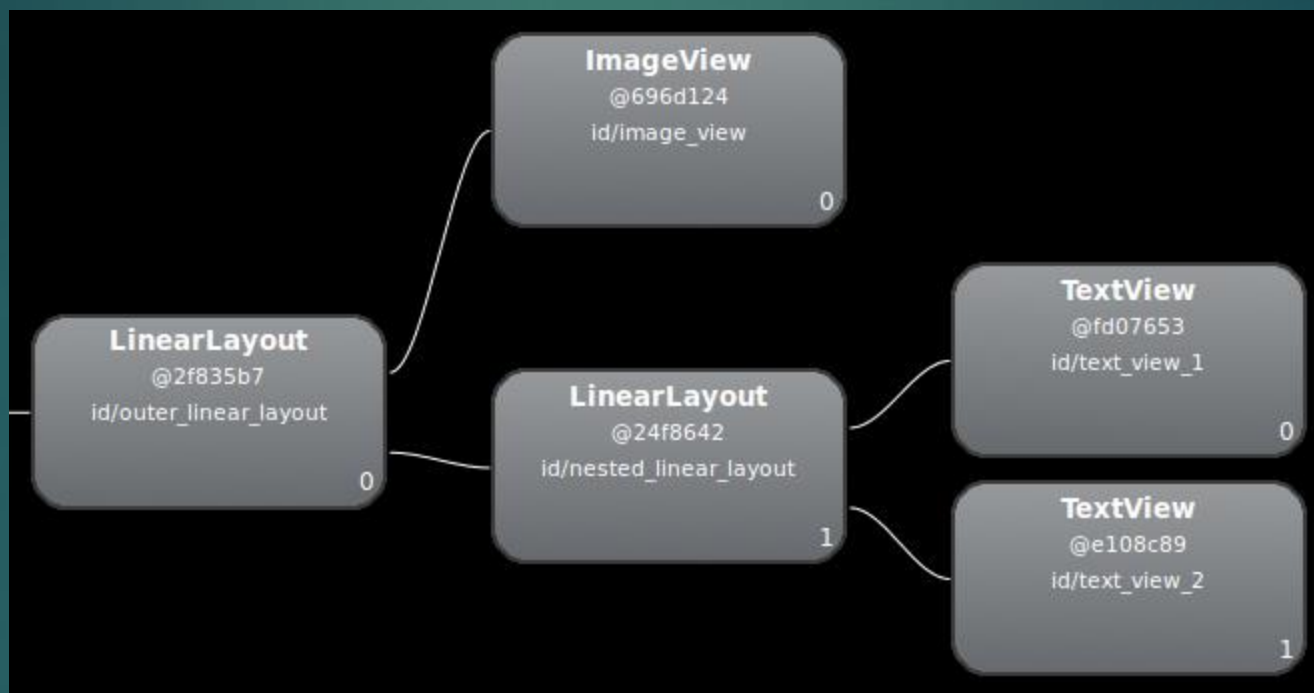
ConstraintLayout



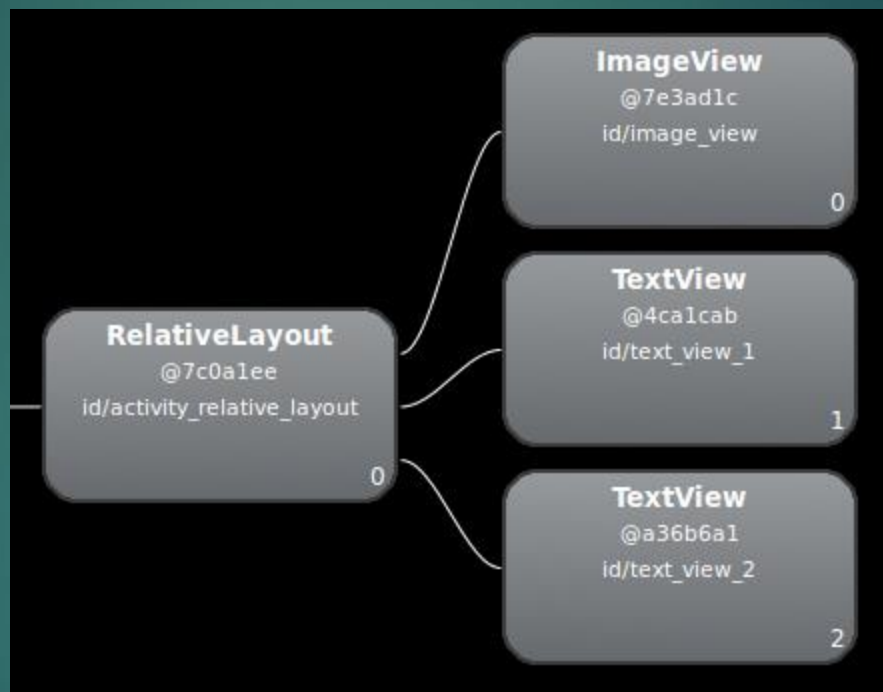
ConstraintLayout



Оптимизация на графичния интерфейс



Оптимизация на графичния интерфейс





Програмиране за мобилни интернет устройства

АКТИВИТИ. СЪЩНОСТ, ЖИЗНЕН ЦИКЪЛ. ИНТЕНТ

Активити. Концепции на активити

- ▶ Приложен компонент, който предоставя екран за потребителско действие
- ▶ Различна концепция от стандартното програмиране – няма main метод
 - ▶ концепция за множество входни точки в програма
 - ▶ ...main activity
 - ▶ Обръщение към съответстващ на състоянието или събитието метод
 - ▶ Концепция за ниска зависимост между отделните активити

Жизнен цикъл

- активно
- в режим на задържане
- спряно - finish()



ИМПЛЕМЕНТАЦИЯ НА ЖИЗНЕНИЯ ЦИКЪЛ

```
public class ExampleActivity extends Activity {  
    @Override  
    public void onCreate(Bundle savedInstanceState) {  
        {  
            super.onCreate(savedInstanceState);  
            // създаване.  
        }  
        @Override  
        protected void onStart() {  
            super.onStart();  
            // стартиране  
        }  
        @Override  
        protected void onResume() {  
            super.onResume();  
            // на фокус  
        }  
        @Override  
        protected void onPause() {  
            super.onPause();  
            // предаване на фокуса  
        }  
        @Override  
        protected void onStop() {  
            super.onStop();  
            // невидимо  
        }  
        @Override  
        protected void onDestroy() {  
            super.onDestroy();  
            // унищожаване  
        }  
    }  
}
```


Методът onCreate()

- ▶ Инициализиращи действия
 - ▶ Изпълнява се само веднъж
 - ▶ Създава екран на базата на параметри от java и xml кода
 - ▶ **protected void** onCreate(Bundle savedInstanceState)
{
 super.onCreate(savedInstanceState);
 setContentView(R.layout.**activity_main**);
}
- ▶ Обектът Bundle
 - ▶ onSaveInstanceState()
 - ▶ onRestoreInstanceState()

Методът onStart()

- ▶ Подготвя екрана за визуализация
 - ▶ Инициализация на код свързан с UI
 - ▶ Много кратък като времетраене

Методът `onResume()`

- ▶ Основния режим на активити
 - ▶ Видимо, интерактивно

Прехващане на събитие

- ▶ В XML изглед
 - ▶ `android:onClick="onClick"/>`
 - ▶ **public void** `onClick(View view)`
- ▶ В Java контролер - listener
- ▶

```
.setOnClickListener(new View.OnClickListener() {  
    @Override  
    public void onClick(View view) {  
        // обработка на събитието  
    }  
});
```

Методът onPause()

- ▶ При загуба на фокус но все още видимо активити
 - ▶ `onStop()`, `onResume()`

Методът onStop()

- ▶ При скриване и поемане на управлението от друго активити

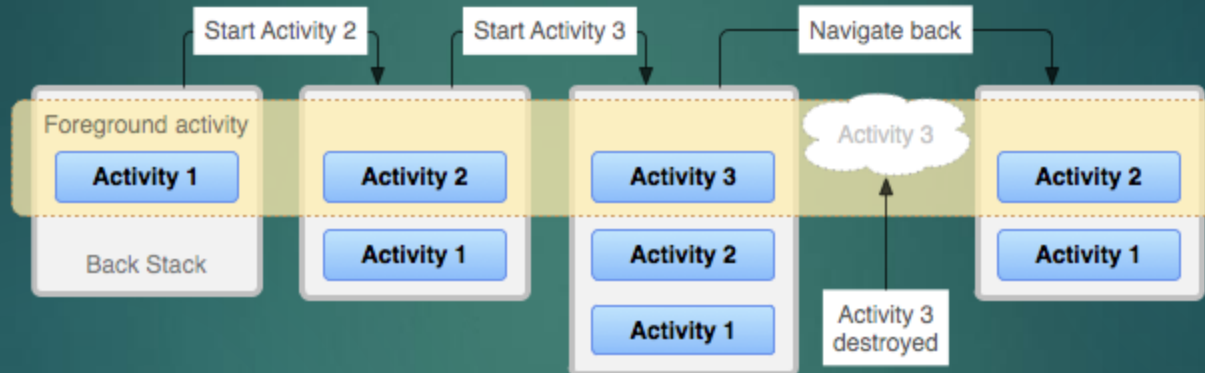
Методът onDestroy()

- ▶ Освобождаване на ресурси

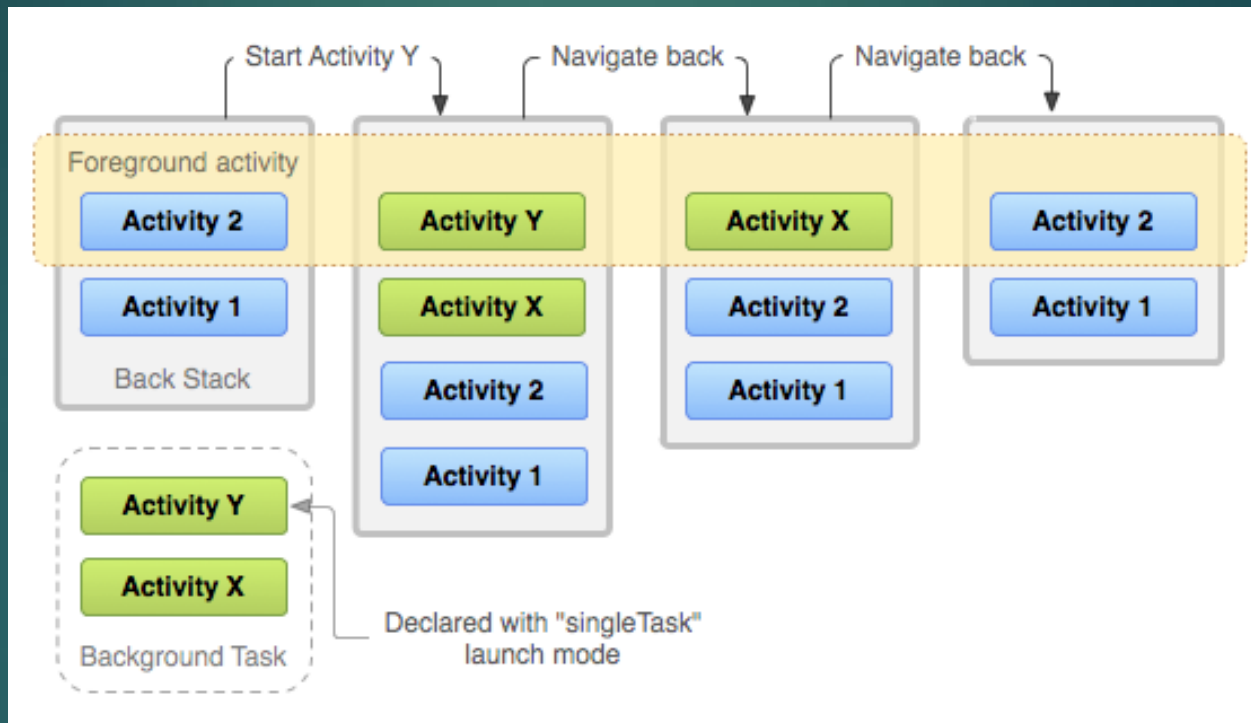
Промяна на конфигурация

- ▶ Промяна на ориентация
 - ▶ Промяна на локализация
 - ▶ Промяна на вход
 - ▶ Др.
-
- ▶ Android 7.0 (API level 24)

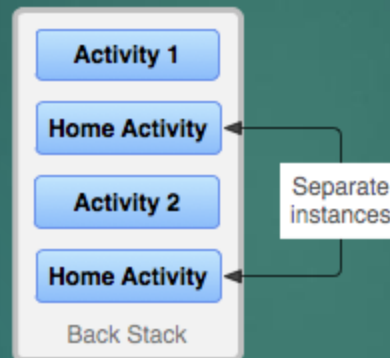
Activity backstack



Задачи и Activity backstack



Задачи и Activity backstack

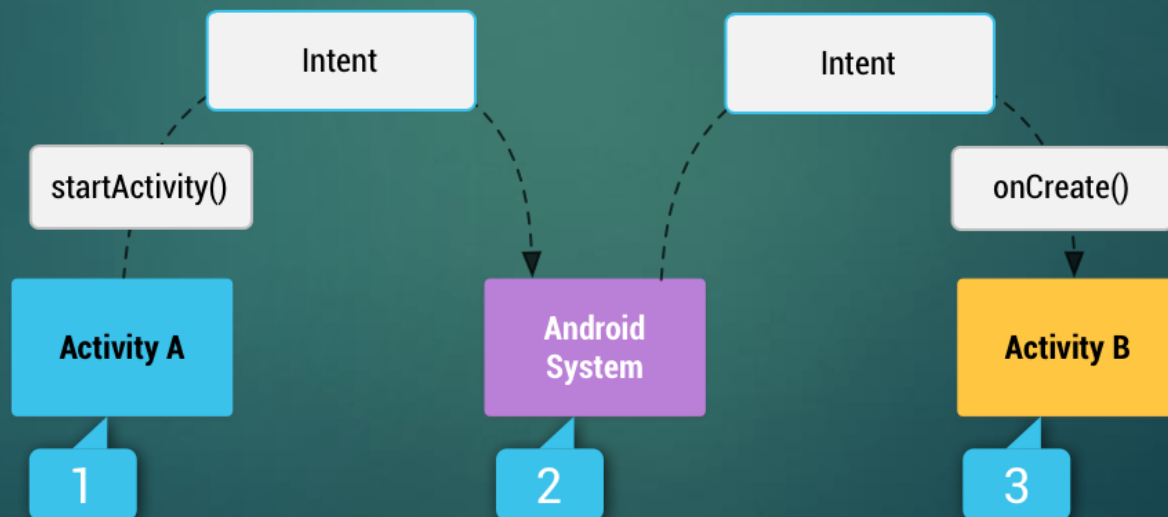


Намерение (Intent)

- ▶ Асинхронен механизъм за активиране на компоненти
 - ▶ Навигация между активности

Навигация между активити

- ▶ Типове Intents
 - ▶ ЕКСПЛИЦИТНИ
 - ▶ ИМПЛИЦИТНИ

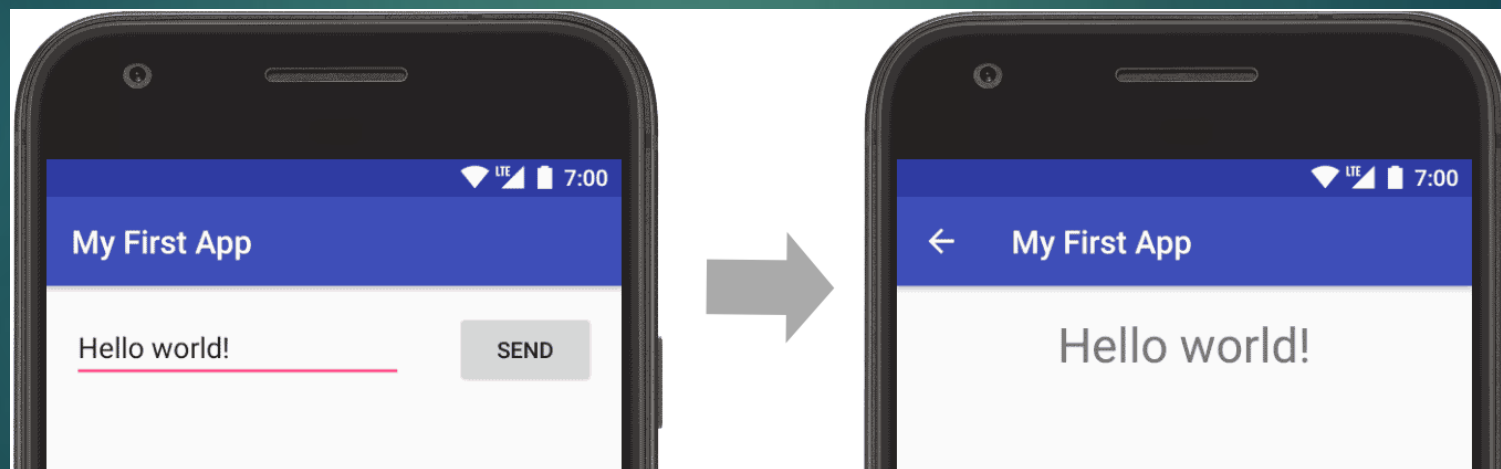


Структура на Intents

- ▶ име на компонента който ще се стартира – експлицитни
- ▶ действие - ACTION_VIEW, ACTION_SEND и др.
- ▶ потребителско дефинирано действие :
 - ▶ `static final String ACTION_TIMETRAVEL = "com.example.action.TIMETRAVEL";`
- ▶ категория – характеристика на обекта който трябва да обработи заявката CATEGORY_BROWSABLE, CATEGORY_LAUNCHER
 - ▶ `addCategory()`
- ▶ допълнителна информация

Навигация между активити. Експлицитен интент

- ▶ Активиране на активити
 - ▶ `startActivity()`,



Навигация между активити. Експлицитен интент

```
public class MainActivity extends AppCompatActivity {  
    public static final String EXTRA_MESSAGE = "com.example.androidmagistri.MESSAGE";  
  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.activity_main);  
        final EditText editText = findViewById(R.id.editText1);  
        Button submit = findViewById(R.id.button);  
        submit.setOnClickListener(new View.OnClickListener() {  
            @Override  
            public void onClick(View view) {  
                Intent intent = new Intent(MainActivity.this, DisplayMessageActivity.class);  
                String message = editText.getText().toString();  
                intent.putExtra(EXTRA_MESSAGE, message);  
                startActivity(intent);  
            }  
        });  
    }  
}
```


Навигация между активити. Експлицитен интент

- ▶ `Intent intent = new Intent(MainActivity.this, DisplayMessageActivity.class);`
 - ▶ `android:onClick`
 - ▶ `Intent intent = new Intent(this, DisplayMessageActivity.class);`
 - ▶ Контекст
 - ▶ Клас
- ▶ `putExtra()`
 - ▶ мап
- ▶ `startActivity()`

Навигация между активити. Експлицитен интент

```
public class DisplayMessageActivity extends AppCompatActivity {  
  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.activity_display_message);  
        Intent intent = getIntent();  
        String message = intent.getStringExtra(MainActivity.EXTRA_MESSAGE);  
        TextView textView = findViewById(R.id.textView4);  
        textView.setText(message);  
    }  
}
```

Навигация между активити. Експлицитен интент

- ▶ Извличане на екстрите
- ▶ `getIntent()`, `getStringExtra()`
 - ▶ В обект или променлива
 - ▶ В bundle
 - ▶ `Intent intent = getIntent();`
`Bundle b = intent.getExtras();`

Навигация между активити. Експлицитен интент

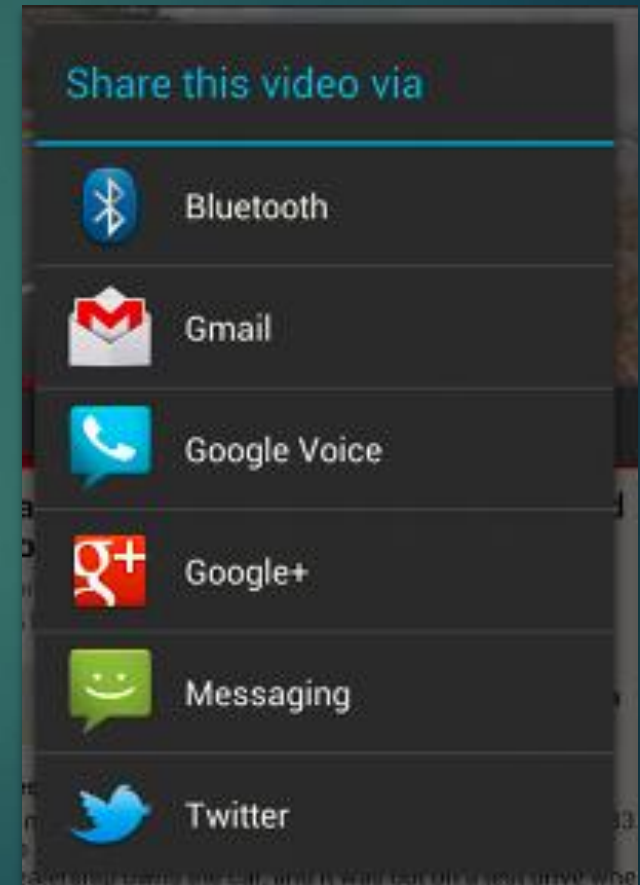
- ▶ `startActivityResult()`
 - ▶ `startActivityResult(intent, 2);`
 - ▶ Интент
 - ▶ Код на заявката
 - ▶ ?
 - ▶ `setResult();`
 - ▶ `finish();`
- ▶ Callback метод
 - ▶ **protected void** `onActivityResult(int requestCode, int resultCode, Intent data)`
 - ▶ `if (requestCode == 2) {`
 - `if (resultCode == RESULT_OK) {`

Навигация между активити. Имплицитен интент

```
Intent sendIntent = new Intent();  
sendIntent.setAction(Intent.ACTION_SEND);  
sendIntent.putExtra(Intent.EXTRA_TEXT, textMessage);  
sendIntent.setType("text/plain");  
if (sendIntent.resolveActivity(getPackageManager()) != null) {  
    startActivity(sendIntent);  
}
```

Навигация между активити. Имплицитен интент с меню за избор

```
Intent sendIntent = new Intent(Intent.ACTION_SEND);  
...  
String title = getResources().getString(R.string.chooser_title);  
Intent chooser = Intent.createChooser(sendIntent, title);  
if (sendIntent.resolveActivity(getPackageManager()) != null) {  
    startActivity(chooser);  
}
```



Навигация между активити. Имплицитен интент. Получаване на интент.

- ▶ Дефиниране на филтри с манифест файла - `<intent-filter>`
 - ▶ `<action>`
 - ▶ `<data>`
 - ▶ `<category>`

```
<activity android:name="ShareActivity">  
<intent-filter>  
  <action android:name="android.intent.action.SEND"/>  
  <category  
android:name="android.intent.category.DEFAULT"/>  
  <data android:mimeType="text/plain"/>  
</intent-filter>  
</activity>
```

Стартиране на услуга чрез интент

- ▶ Стартиране на услуга
 - ▶ `startService()`, `bindService()`
- ▶ Инициране на broadcast
 - ▶ `sendBroadcast()`, `sendOrderedBroadcast()`,
`sendStickyBroadcast()`

Pending intent

- ▶ Взаимодействие с други приложения
- ▶ Споделяне на права

Управление на интенти

- ▶ Android 5.0 (от API level 21) JobScheduler



Програмиране за мобилни интернет устройства

ИНТЕРФЕЙСЪТ АДАПТЕР. ИЗГЛЕДИ СВЪРЗАНИ С
ИЗПОЛЗВАНЕТО НА АДАПТЕР

Проблем

- ▶ работа с данни
- ▶ трансформация на данни
 - ▶ Базы данни, колекции, контейнери..

Адаптер

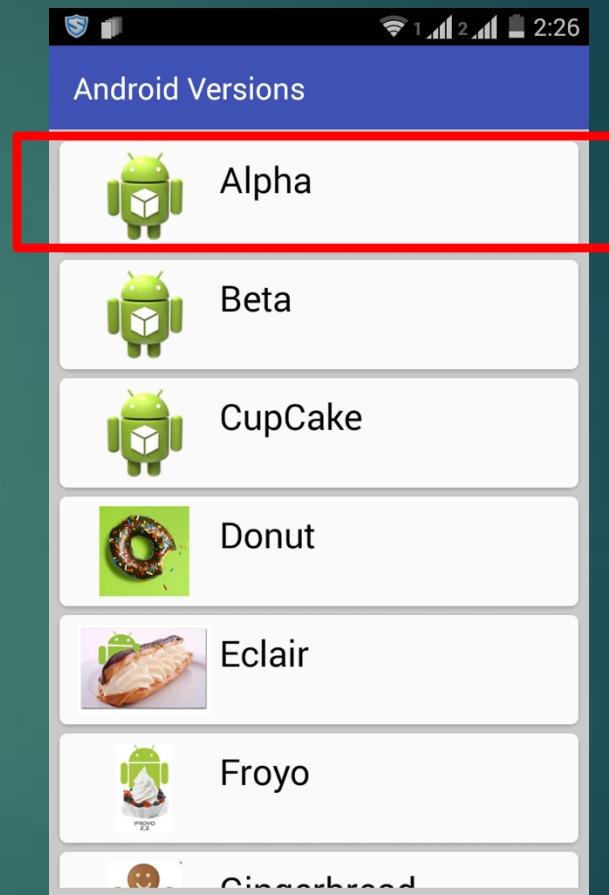
- ▶ Adapter е клас, който служи за свързване на данни с елементите в потребителския интерфейс.
- ▶ **Роля на Adapter:**
 - Осигурява данни за всеки елемент в списъка.
 - Създава изгледи за всеки елемент и ги свързва с данните.
 - Установява връзката между данните и потребителския интерфейс.

Принцип на работа



СЪЩНОСТ

- ▶ Организация на елементите от изгледа
 - ▶ „Оживяване“ на елемент
 - ▶ Организация в изглед
 - ▶ Прихващане на събитие
 - ▶ Кеширане на изгледи



Типове адаптери

- ▶ BaseAdapter
 - ▶ Базов клас за създаване на адаптери
- ▶ ArrayAdapter
 - ▶ При източник на информация от масив
 - ▶ Подклас на BaseAdapter
- ▶ SimpleAdapter
 - ▶ При работа със статични данни – един запис от колекцията един ред от изгледа
- ▶ Custom Adapter
 - ▶ При необходимост от реализация на специфични изисквания
 - ▶ Всички типове адаптери
- ▶ PagerAdapter (...)

Създаване на адаптер

- ▶ `ArrayAdapter(Context context, int resource, int textViewResourceId, T[] objects)`
 - ▶ `context`
 - ▶ Референция към текущото активити (`this`)
 - ▶ `getApplicationContext()` – за активити
 - ▶ `getActivity()` – за фрагмент
 - ▶ `resource`
 - ▶ Референция към layout който искаме да се зареди – `ListView`
 - ▶ `textViewResourceId`
 - ▶ Идентификация на отделните `TextView`
 - ▶ `objects`
 - ▶ Източника на информация
- ▶

```
String animalList[] = {"Lion","Tiger","Monkey","Elephant","Dog","Cat","Camel"};
ArrayAdapter arrayAdapter = new ArrayAdapter(this, R.layout.list_view_items,
R.id.textView, animalList);
```

RecyclerView

- ▶ компонент за показване на големи набори от данни в списъчен или мрежов вид, като използва Adapter, за да свърже данните с изгледите
- ▶ **Предимства на RecyclerView:**
 - ▶ Повторно използване на изгледи за оптимизация.
 - Поддръжка за различни LayoutManagers, като LinearLayoutManager и GridLayoutManager.
 - Добавяне на анимации и декоратори.

activity_main.xml

```
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">
```

```
<ListView
    android:id="@+id/simpleListView"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:divider="#000"
    android:dividerHeight="2dp"/>
```

```
</RelativeLayout>
```

activity_list_view.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">

    <TextView
        android:id="@+id/textView"
        android:layout_width="fill_parent"
        android:layout_height="wrap_content"
        android:layout_gravity="center"
        android:padding="@dimen/activity_horizontal_margin"
        android:textColor="#000" />

</LinearLayout>
```

MainActivity.java

```
import android.support.v7.app.AppCompatActivity;
import android.os.Bundle;
import android.widget.ArrayAdapter;
import android.widget.ListView;

public class MainActivity extends AppCompatActivity {
    ListView simpleList;
    String animalList[] =
{"Lion","Tiger","Monkey","Elephant","Dog","Cat","Camel"};
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        simpleList = (ListView) findViewById(R.id.simpleListView);

        ArrayAdapter<String> arrayAdapter = new ArrayAdapter<String>(this,
R.layout.activity_list_view, R.id.textView, animalList);
        simpleList.setAdapter(arrayAdapter);
    }

}
}
```

Прихващане на събитие

```
private OnItemClickListener messageClickedHandler = new
ItemClickListener() {

    public void onItemClick(AdapterView parent, View v, int position, long id) {

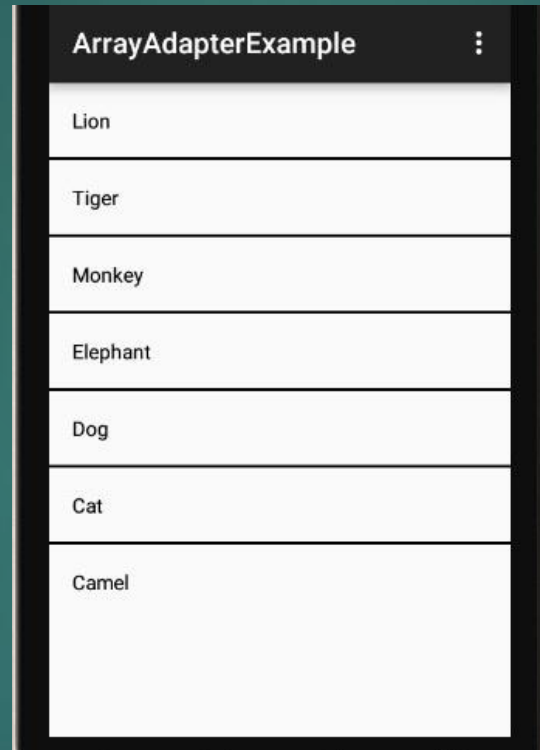
        // програмна логика

    }

};

listView.setOnItemClickListener(messageClickedHandler);
```

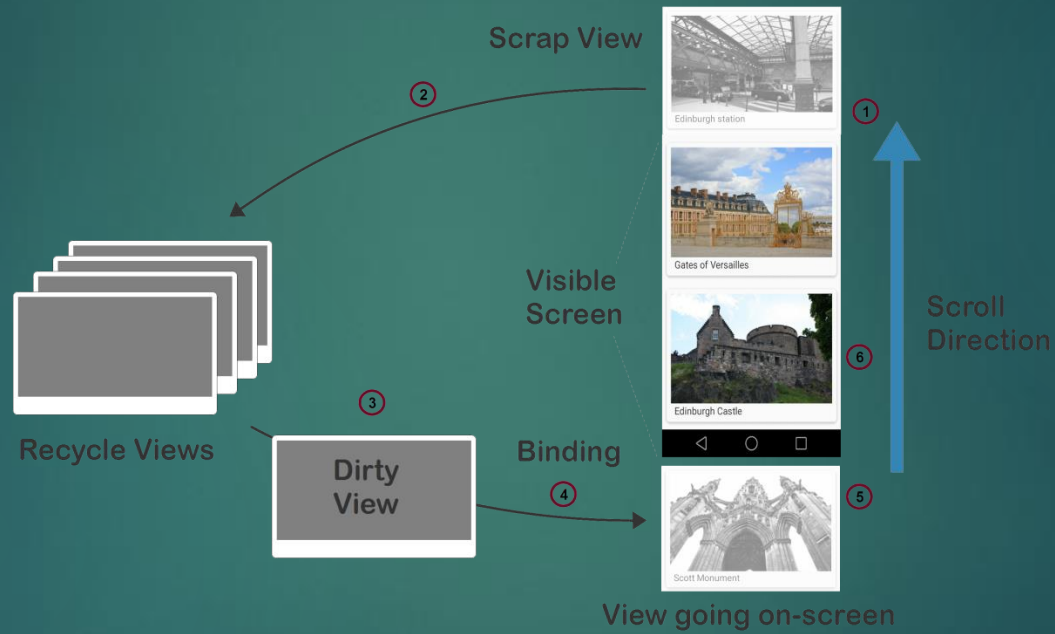
Създаване на адаптер



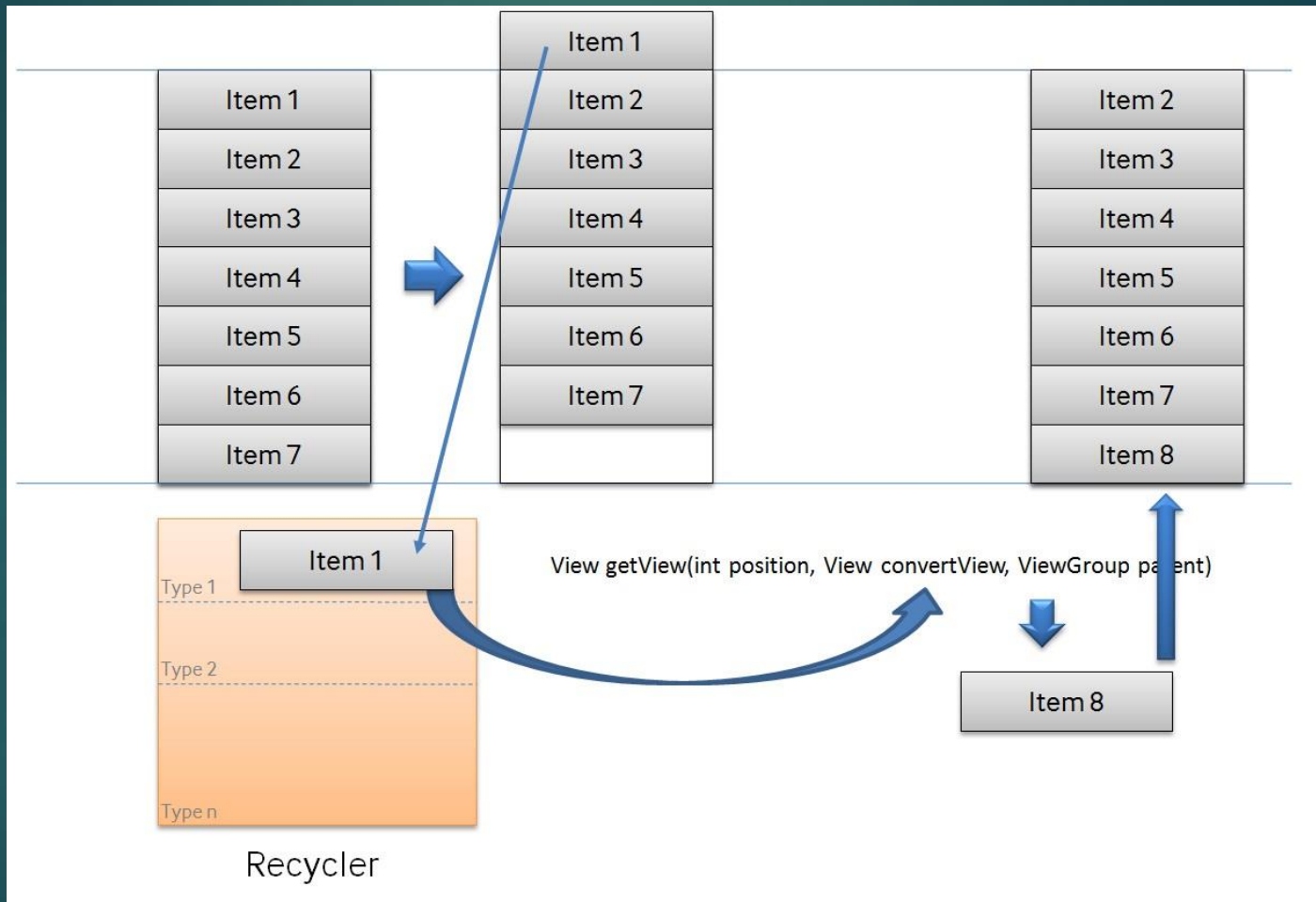
Изгледи работещи с адаптери

- ▶ RecyclerView
- ▶ CardView
- ▶ PagerView

RecyclerView



RecyclerView



Добавяне на RecyclerView в XML

```
<RecyclerView  
    android:id="@+id/recyclerView"  
    android:layout_width="match_parent"  
    android:layout_height="match_parent"/>
```

Настройка на RecyclerView в Activity

- ▶ **LayoutManager:** Определя начина на подредба (линеен или мрежов).
- ▶ **Adapter:** Свързва се с RecyclerView, за да подаде данните.

Настройка на Адаптер в Activity

```
public class MyAdapter extends RecyclerView.Adapter<MyAdapter.ViewHolder> {  
    private List<String> dataList;  
    private View.OnClickListener itemClickListener;  
  
    public MyAdapter(List<String> dataList, View.OnClickListener itemClickListener) {  
        this.dataList = dataList;  
        this.itemClickListener = itemClickListener;  
    }  
}
```

Настройка на Адаптер в Activity

```
public static class ViewHolder extends RecyclerView.ViewHolder {  
    public TextView textView;  
  
    public ViewHolder(View itemView) {  
        super(itemView);  
        textView = itemView.findViewById(R.id.textView);  
    }  
}  
  
@Override  
public ViewHolder onCreateViewHolder(ViewGroup parent, int viewType) {  
    View view = LayoutInflater.from(parent.getContext()).inflate(R.layout.item_layout,  
parent, false);  
    return new ViewHolder(view);  
}
```

Настройка на Адаптер в Activity

```
@Override
```

```
public void onBindViewHolder(ViewHolder holder, int position) {
```

```
    holder.textView.setText(dataList.get(position));
```

```
    holder.itemView.setOnClickListener(itemClickListener);
```

```
}
```

```
@Override
```

```
public int getItemCount() {
```

```
    return dataList.size();
```

```
}
```

```
}
```

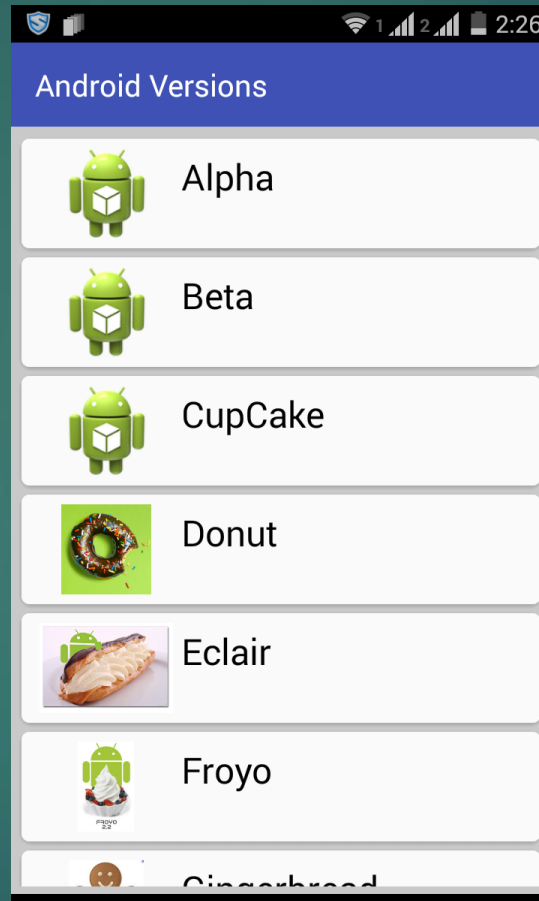
Настройка на RecyclerView в Activity

```
public class MainActivity extends AppCompatActivity {  
  
    private RecyclerView recyclerView;  
  
    private MyAdapter adapter;  
  
    private List<String> dataList;  
  
    @Override  
  
    protected void onCreate(Bundle savedInstanceState) {  
  
        super.onCreate(savedInstanceState);  
  
        setContentView(R.layout.activity_main);  
  
  
        recyclerView = findViewById(R.id.recyclerView);  
  
        recyclerView.setLayoutManager(new LinearLayoutManager(this));  
  
  
        dataList = new ArrayList<>();  
        for (int i = 1; i <= 20; i++) {  
            dataList.add("Item " + i);  
        }  
    }  
}
```


Настройка на **RecyclerView** в Activity

```
adapter = new MyAdapter(dataList, new View.OnClickListener() {  
    @Override  
    public void onClick(View view) {  
        int position = recyclerView.getChildAdapterPosition(view);  
        String item = dataList.get(position);  
        Toast.makeText(MainActivity.this, "Clicked: " + item, Toast.LENGTH_SHORT).show();  
    }  
});  
  
recyclerView.setAdapter(adapter);  
}  
}
```

RecyclerView



Пример за GridLayoutManager

```
recyclerView.setLayoutManager(new GridLayoutManager(this, 2));
```

- ▶ Добавяне на разделител

- ▶ `recyclerView.addItemDecoration(new DividerItemDecoration(this, LinearLayoutManager.VERTICAL));`



Програмиране за мобилни Интернет устройства

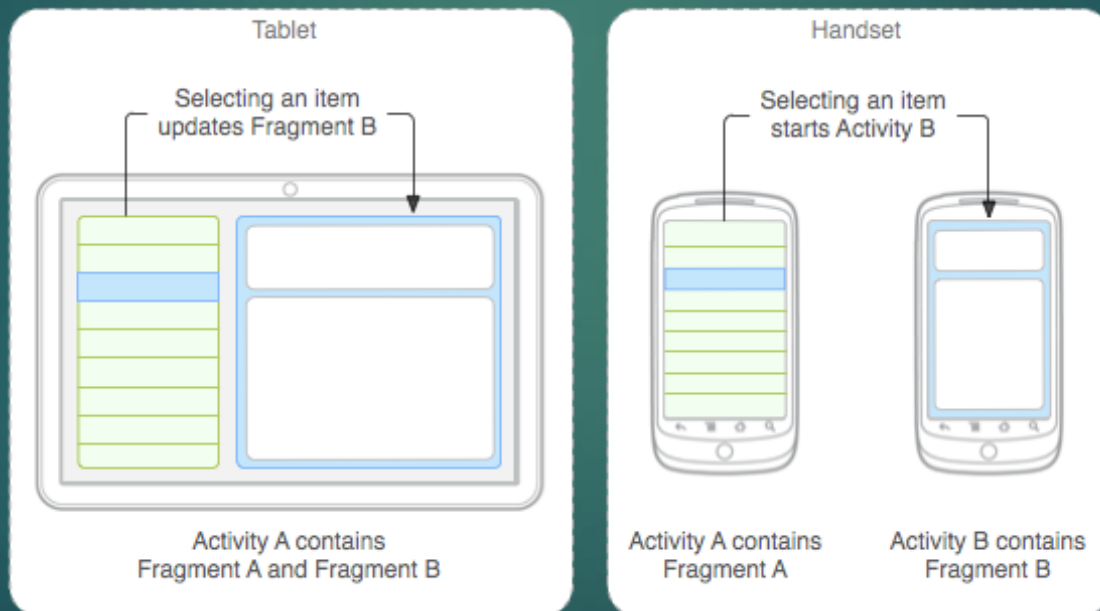
ФРАГМЕНТ. ЖИЗНЕН ЦИКЪЛ НА ФРАГМЕНТ.
УПРАВЛЕНИЕ И ВЗАИМОДЕЙСТВИЯ НА ФРАГМЕНТИ

Проблеми на Активити

- ▶ Проблем с размера на екрана
- ▶ Проблем при промяна на конфигурация

Фрагменти

- ▶ Android 3.0 (API level 11) – в отговор на появата на мобилни устройства с големи екрани
- ▶ Възможност за промяна на изглед и поведение по време на работа на приложението



Фрагмент. Същност

- ▶ Представява част от поведението и графичния интерфейс на едно активити
 - ▶ Задължително част от активити
- ▶ В комбинация – реализация на по сложни активити компоненти
- ▶ Добавя се към View групата на активиралото го активити
 - ▶ Дефинира и собствен изглед или група от изгледи
 - ▶ Добавя се посредством декларация на `<fragment>` в схемата на изгледа или от Java кода на приложението чрез добавяне към текущ дефиниран изглед

Фрагмент. Същност

- ▶ Концепция на модула – капсулиране и сепарация
 - ▶ Самостоятелен жизнен цикъл **в рамките на активити**
 - ▶ Собствено прихващане и обработване на събития
 - ▶ Възможност за активиране (прикачване) и деактивиране (откачане) докато съдържащото го активити работи
 - ▶ Концепция за преизползване на код
 - ▶ Един фрагмент може да се активира от различни активити

Създаване на фрагменти

- ▶

```
public static class ExampleFragment extends Fragment {  
    @Override  
    public View onCreateView(LayoutInflater inflater, ViewGroup  
    container, Bundle savedInstanceState) {  
        return inflater.inflate(R.layout.example_fragment, container, false);  
    }  
}
```
- ▶ Методът *inflate*

ДОПЪЛНИТЕЛНИ КЛАСОВЕ

- ▶ DialogFragment
 - ▶ Аналогия с dialog helper, но добавя в стека на фрагментите
- ▶ ListFragment
 - ▶ При работа с адаптери
- ▶ PreferenceFragmentCompat
 - ▶ За визуализация на обекти от тип Preference

Добавяне на фрагмент.

Статичен подход

```
► <?xml version="1.0" encoding="utf-8"?>
  <LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="horizontal"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <fragment android:name="com.example.news.ArticleListFragment"
      android:id="@+id/list"
      android:layout_weight="1"
      android:layout_width="0dp"
      android:layout_height="match_parent" />
    <fragment android:name="com.example.news.ArticleReaderFragment"
      android:id="@+id/viewer"
      android:layout_weight="2"
      android:layout_width="0dp"
      android:layout_height="match_parent" />
  </LinearLayout>
```

Добавяне на фрагмент. Статичен подход

```
<androidx.fragment.app.FragmentContainerView  
    xmlns:android="http://schemas.android.com/apk/res/android"  
    android:id="@+id/fragment_container_view"  
    android:layout_width="match_parent"  
    android:layout_height="match_parent"  
    android:name="com.example.ExampleFragment" />
```

Добавяне на фрагмент.

Динамичен подход

- ▶ По време на изпълнение
 - ▶ Необходимост от дефиниране на placeholder – ViewGroup
 - ▶ `<androidx.fragment.app.FragmentContainerView`
xmlns:android="http://schemas.android.com/apk/res/android"
android:id="@+id/fragment_container_view"
android:layout_width="match_parent"
android:layout_height="match_parent" />
- ▶ Принцип на транзакциите – добавяне, премахване, подмяна
 - ▶ API – `FragmentManager`

Добавяне на фрагмент.

Динамичен подход

```
► public class ExampleActivity extends AppCompatActivity {  
    public ExampleActivity() {  
        super(R.layout.example_activity);  
    }  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        if (savedInstanceState == null) {  
            getSupportFragmentManager().beginTransaction()  
                .setReorderingAllowed(true)  
                .add(R.id.fragment_container_view, ExampleFragment.class,  
null)  
                .commit();  
        }  
    }  
}
```

При необходимост от входна информация

```
▶ public class ExampleActivity extends AppCompatActivity {  
    public ExampleActivity() {  
        super(R.layout.example_activity);  
    }  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        if (savedInstanceState == null) {  
            Bundle bundle = new Bundle();  
            bundle.putInt("some_int", 0);  
  
            getSupportFragmentManager().beginTransaction()  
                .setReorderingAllowed(true)  
                .add(R.id.fragment_container_view, ExampleFragment.class,  
bundle)  
                .commit();  
        }  
    }  
}
```

При необходимост от входна информация

```
▶ class ExampleFragment extends Fragment {  
    public ExampleFragment() {  
        super(R.layout.example_fragment);  
    }  
  
    @Override  
    public void onViewCreated(@NonNull View view, Bundle  
savedInstanceState) {  
        int someInt = requireArguments().getInt("some_int");  
        ...  
    }  
}
```


Транзакции на фрагментите. FragmentManager

- ▶ FragmentTransaction API
- ▶ Поредица от промени които трябва да са атомарни
 - ▶ add(), remove(), replace()
- ▶ addToBackStack() !!!
- ▶ commit()
 - ▶ Асинхронен
 - ▶ commitNow() – несъвместим с addToBackStack()
 - ▶ executePendingTransactions()
- ▶ Комплексни операции
 - ▶ Преди commit()
- ▶ Последователност на фрагментите

Замяна на фрагмент

```
FragmentManager fragmentManager = getSupportFragmentManager();  
fragmentManager.beginTransaction()  
    .replace(R.id.fragment_container, ExampleFragment.class, null)  
    .setReorderingAllowed(true)  
    .addToBackStack(null)  
    .commit();
```

...

```
ExampleFragment fragment =  
    (ExampleFragment)  
fragmentManager.findFragmentById(R.id.fragment_container);
```

Управление на фрагменти. FragmentManager

- ▶ `getSupportFragmentManager()`
- ▶ `findFragmentById()` – фрагмент с UI
- ▶ `findFragmentByTag()` – фрагмент без UI (headless fragment)
- ▶ `popBackStack()` – симулиране на Back

Взаимодействие с изгледите

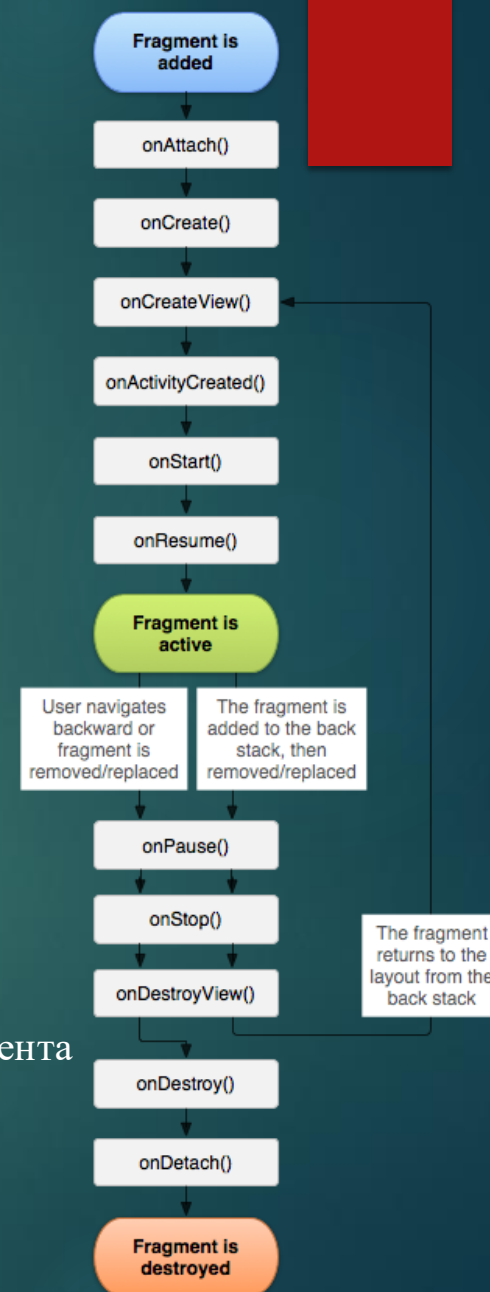
- ▶ detach()
 - ▶ STOPPED
- ▶ attach()

Жизнен цикъл на фрагмент

- ▶ INITIALIZED
- ▶ CREATED
- ▶ STARTED
- ▶ RESUMED
- ▶ DESTROYED

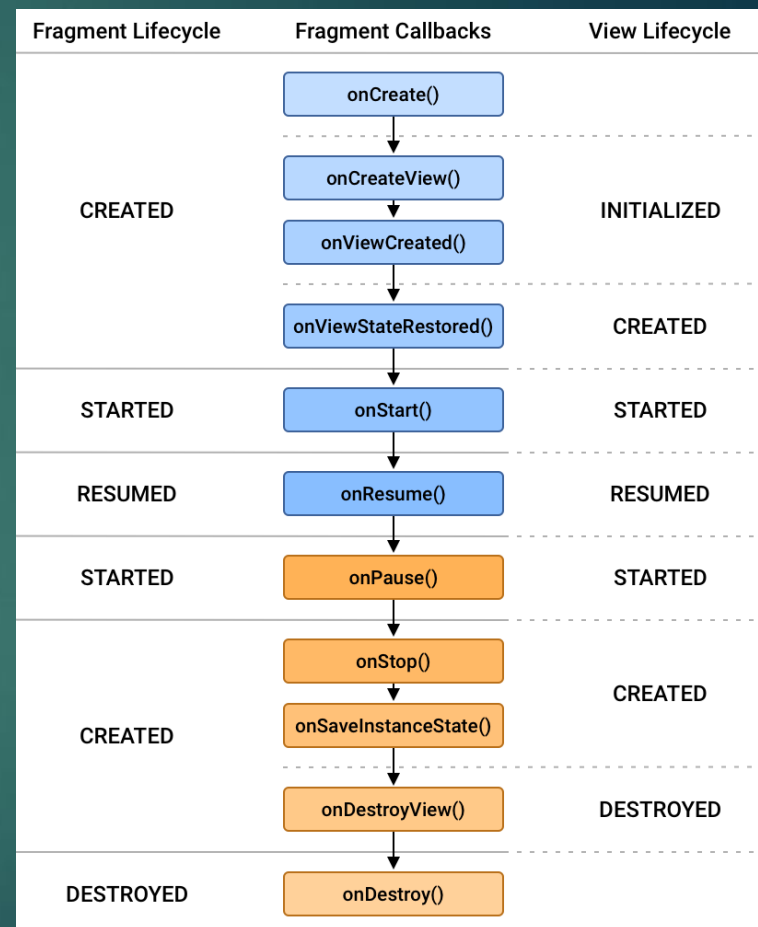
- ▶ onCreate(), onStart(), onResume(), onPause(), onStop(), onDestroy()

- ▶ onCreate() – аналогия с метода на активити
- ▶ onCreateView() – подготвяне и визуализация на фрагмента
- ▶ onPause() – извиква се при излизане от фрагмента
 - ▶ Не е задължително свързано с унищожаването на фрагмента
 - ▶ Запазване на данни

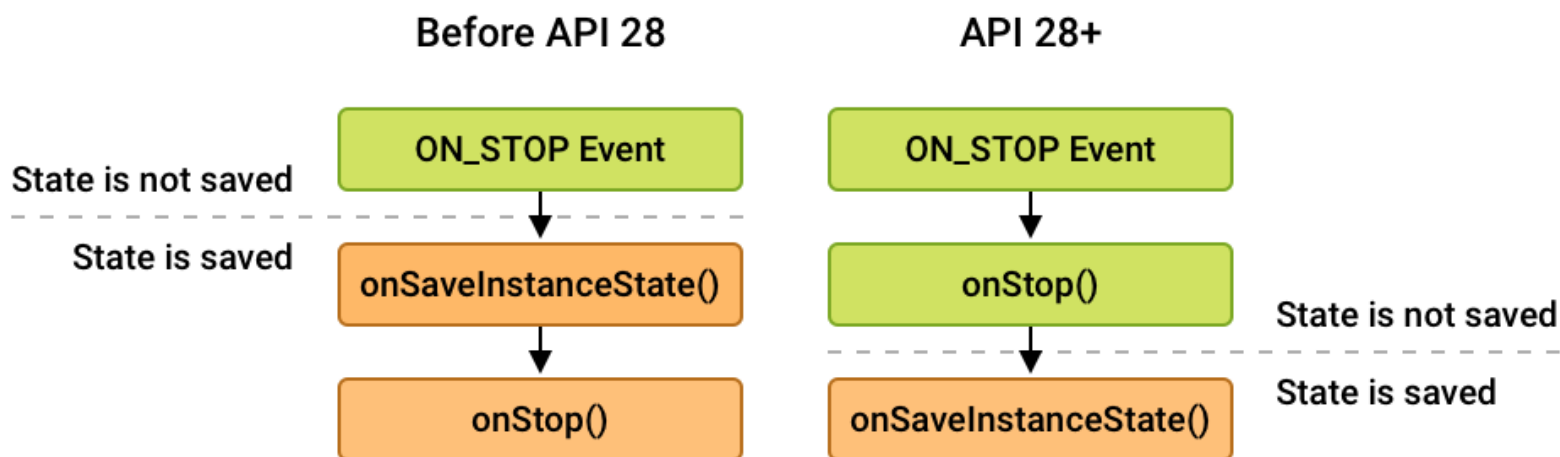


Жизнен цикъл на фрагмент

- ▶ Директно отражение на методите от Активити
- ▶ `onAttach()`
- ▶ `onCreateView()` – за създаване на йерархията от изгледи асоциирани с фрагмента
- ▶ `onActivityCreated()` – при повикване на `onCreate()` на съдържащото Активити
- ▶ `onDestroyView()` – при премахване на йерархията от изгледи
- ▶ `onDetach()` – при премахване на асоциацията между Активити и Фрагмента



Жизнен цикъл на фрагмент



КОМУНИКАЦИЯ С АКТИВИТИ

- ▶ Фрагмент -> Активити

- ▶ `View listView = getActivity().findViewById(R.id.list);`

- ▶ Активити -> Фрагмент

- ▶ `ExampleFragment fragment = (ExampleFragment) getSupportFragmentManager().findFragmentById(R.id.example_fragment);`

Споделяне на информация между Активити и Фрагмент

- ▶ Необходимост от предаване на данни между програмните елементи
 - ▶ ViewModel (LiveData)
 - ▶ Callback интерфейс
 - ▶ имплементиран от активити

Споделяне на информация между Активити и Фрагмент. ViewModel подход

```
▶ public class MainActivity extends AppCompatActivity {  
    private ItemViewModel viewModel;  
  
    @Override  
    public void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        viewModel = new ViewModelProvider(this).get(ItemViewModel.class);  
        viewModel.getSelectedItem().observe(this, item -> {  
            // взаимодействие с item  
        });  
    }  
}
```

Споделяне на информация между Активити и Фрагмент. ViewModel ПОДХОД

```
▶ public class ListFragment extends Fragment {  
    private ItemViewModel viewModel;  
  
    @Override  
    public void onViewCreated(@NonNull View view, Bundle  
savedInstanceState) {  
        super.onViewCreated(view, savedInstanceState);  
        viewModel = new  
ViewModelProvider(requireActivity()).get(ItemViewModel.class);  
  
        ...  
  
        items.setOnClickListener(item -> {  
            viewModel.select(item);  
        });  
    }  
}
```



Програмиране за мобилни интернет устройства

ИЗГРАЖДАНЕ НА ГРАФИЧЕН ИНТЕРФЕЙС С
КОТЛИН. JETPACK COMPOSE

Недостатъци на Класическия подход

- ▶ Сложност и четимост на кода
- ▶ Трудности при управление на състоянието
- ▶ Зависимост от XML и Activity/Fragment код
- ▶ Трудности при използване на динамични UI компоненти
 - ▶ много различни View-и и адаптери
- ▶ Ограничени възможности за повторно използване на компоненти

Jetpack Compose

- ▶ позволява създаване на UI компоненти директно чрез код - декларативен подход
- ▶ премахва нуждата от XML файлове и улеснява управлението на състояния
- ▶ Поддържа **State** - улеснява обновяването на UI при промени в данните

Jetpack Compose

- ▶ декларативното програмиране - позволява да се определи как трябва да изглежда UI при дадено състояние, вместо да се пишат конкретни инструкции за това как да се нарисува и подреди всеки елемент
 - ▶ Вместо да обновяваме даден `TextView` при промяна на текст, в Compose просто задаваме новото състояние, а интерфейсът автоматично се актуализира

Jetpack Compose

```
@Composable
fun CounterExample() {
    var count by remember { mutableStateOf(0) }

    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(16.dp),
        verticalArrangement = Arrangement.Center
    ) {
        Text(
            text = "Count: $count",
            fontSize = 24.sp,
            modifier = Modifier.padding(bottom = 16.dp)
        )

        Button(onClick = { count++ }) {
            Text("Increase Count")
        }
    }
}

@Preview(showBackground = true)
@Composable
fun PreviewCounterExample() {
    CounterExample()
}
```


@Composable

@Composable

```
fun Greeting(name: String) {  
    Text(text = "Hello, $name!")  
}
```

- ▶ @Composable - специфична за Jetpack Compose
 - ▶ позволява на функцията да участва в рендирането на интерфейса, като информира Compose да се отнася към нея като част от UI дървото.

Интегриране с останалата част от Android

- ▶ Jetpack Compose е проектиран да работи безпроблемно с традиционните Android компоненти като View и Fragment.
- ▶ Съществува **ComposeView**, който позволява вмъкване на Compose UI в съществуващите XML Layout-и, както и използването на традиционните View компоненти в Jetpack Compose

ОСНОВНИ КОМПОНЕНТИ В Jetpack Compose

► Text

- Използва се за показване на текстови елементи.

► @Composable

```
fun GreetingText() {  
    Text(text = "Hello, Compose!", fontSize = 20.sp, color = Color.Blue)  
}
```

► Image

► @Composable

```
fun ProfileImage() {  
    Image(painter = painterResource(R.drawable.profile_picture),  
        contentDescription = "Profile Image")  
}
```

ОСНОВНИ КОМПОНЕНТИ В Jetpack Compose

- ▶ Button

- ▶ Класически бутон, който поддържа **onClick** за интеракции

- ▶ @Composable

```
fun ClickableButton() {  
    Button(onClick = { /* действие при клик */ }) {  
        Text("Click Me")  
    }  
}
```

Row и Column

- ▶ Използват се за подреждане на елементи съответно по хоризонтала и вертикала
- ▶ Предоставят гъвкавост за организиране на елементи по лесен и интуитивен начин

- ▶ @Composable

```
fun LayoutExample() {  
    Column(modifier = Modifier.padding(16.dp)) {  
        Text("Item 1")  
        Text("Item 2")  
        Text("Item 3")  
    }  
}
```

Scaffold

- ▶ Scaffold е контейнер за структура на екрани с елементи като TopAppBar, BottomNavigation, FloatingActionButton, и Drawer

- ▶ @Composable

```
fun MainScreen() {  
    Scaffold(  
        topBar = { TopAppBar(title = { Text("My App") }) },  
        floatingActionButton = {  
            FloatingActionButton(onClick = { /* действие */ }) {  
                Icon(Icons.Filled.Add, contentDescription = "Add")  
            }  
        }  
    ) { padding ->  
        Content(modifier = Modifier.padding(padding))  
    }  
}
```

Модификатори (Modifiers)

- ▶ Модификаторите в Jetpack Compose са инструмент за добавяне на стилизиране и функционалност към Composable елементите. Например:
 - ▶ `Padding`: Добавя вътрешен отстъп около елемент.
 - ▶ `Background`: Променя фона на елемент.
 - ▶ `Size`: Определя размера на елемента.
 - ▶ `clickable`: Позволява да добавим клик събитие към елемент.

Модификатори (Modifiers)

@Composable

```
fun StyledText() {  
    Text(  
        text = "Hello, Jetpack Compose!",  
        modifier = Modifier  
            .padding(16.dp)  
            .background(Color.LightGray)  
            .fillMaxWidth()  
            .padding(8.dp),  
        color = Color.Black  
    )  
}
```


Управление на цветовете в Jetpack Compose

```
@Composable
fun ColoredText() {
    Text(
        text = "Colored Text",
        color = Color.Blue,
        modifier = Modifier.padding(16.dp)
    )
}
```

Оформление с Card

@Composable

```
fun CardExample() {
```

```
    Card(elevation = 4.dp, modifier = Modifier.padding(8.dp)) {
```

```
        Text("This is inside a card")
```

```
    }
```

```
}
```

Анимации в Jetpack Compose

- ▶ Поддържа лесно използване на анимации, като анимиране на размери, позиции и цветове.
- ▶ Някои основни функции за анимации включват `animateDpAsState`, `animateColorAsState`, и `Crossfade`.
- ▶ `@Composable`

```
fun AnimatedBox() {  
    var expanded by remember { mutableStateOf(false) }  
    val size by animateDpAsState(targetValue = if (expanded) 200.dp else 100.dp)  
  
    Box(  
        modifier = Modifier  
            .size(size)  
            .background(Color.Blue)  
            .clickable { expanded = !expanded }  
    )  
}
```

Предварителен преглед (Preview) на Composable функции

- ▶ предварителен преглед на Composable функциите в Android Studio чрез анотацията @Preview.
- ▶ Това улеснява разработчиците да видят как ще изглежда интерфейсьт, без да стартират приложението на емулятор
- ▶ @Preview

```
@Composable
```

```
fun PreviewGreeting() {
```

```
    Greeting(name = "Compose")
```

```
}
```

Jetpack Compose

- ▶ Compose наблюдава промените в състоянието и автоматично актуализира само тези части на интерфейса, които са пряко засегнати от промяната

Управление на състоянието (State Management)

- ▶ В декларативния подход, при промяна на състоянието (например текст или стойност на брояч), Jetpack Compose автоматично обновява свързаните UI компоненти.
- ▶ **MutableState** и **remember** се използват за управление на състоянието в Composable функции.

- ▶ @Composable

```
fun Counter() {  
    var count by remember { mutableStateOf(0) }  
    Text(text = "Counter: $count")  
    Button(onClick = { count++ }) {  
        Text("Increase")  
    }  
}
```

Сложни Layout-и с ConstraintLayout

► Подобен на XML-базирания ConstraintLayout

► @Composable

```
fun ComplexLayout() {  
    ConstraintLayout {  
        val (button, text) = createRefs()  
  
        Button(onClick = { /* TODO */ }, modifier = Modifier.constrainAs(button) {  
            top.linkTo(parent.top) }) {  
            Text("Button")  
        }  
  
        Text("Hello", Modifier.constrainAs(text) { top.linkTo(button.bottom) })  
    }  
}
```



Програмиране за мобилни интернет устройства

ПРОЦЕСИ И НИШКИ. ПАРАЛЕЛНА, АСИНХРОННА
РАБОТА И ОБРАБОТКА НА ИНФОРМАЦИЯ ВЪВ
ФОНОВ РЕЖИМ.

Концепция на паралелната обработка

- ▶ Последователно изпълнение – ограничения в хардуера
 - ▶ Един общ процес
 - ▶ Ниска производителност
- ▶ Съвременния хардуер
 - ▶ Многопроцесорен
 - ▶ Многоядрен
- ▶ Ефективност на изпълнение и производителност
 - ▶ Ефективно използване на хардуера
 - ▶ Използване на паралелни блокове на обработка

Процеси и нишки

- ▶ Процес
 - ▶ Съдържание на процеса – process context
 - ▶ Недостатък
- ▶ Нишка

Нишки. Същност, предимства и недостатъци

- ▶ „Лек“ процес – подпроцес в рамките на създаващият нишката процес
- ▶ Достъп до контекста на процеса
 - ▶ Бързодействие
 - ▶ Критични секции
 - ▶ Синхронизация – synchronized
 - ▶ Голямо влияние върху изпълнението и производителността
 - ▶ Ефективно използване
 - ▶ Принцип на най-малкия блок
- ▶ volatile

Проблеми при паралелна обработка

- ▶ Взаимна блокировка – deadlock
- ▶ Неконсистентност на данните (thread safe problem)
- ▶ Квази паралелност – гладни нишки

Нишки в Java

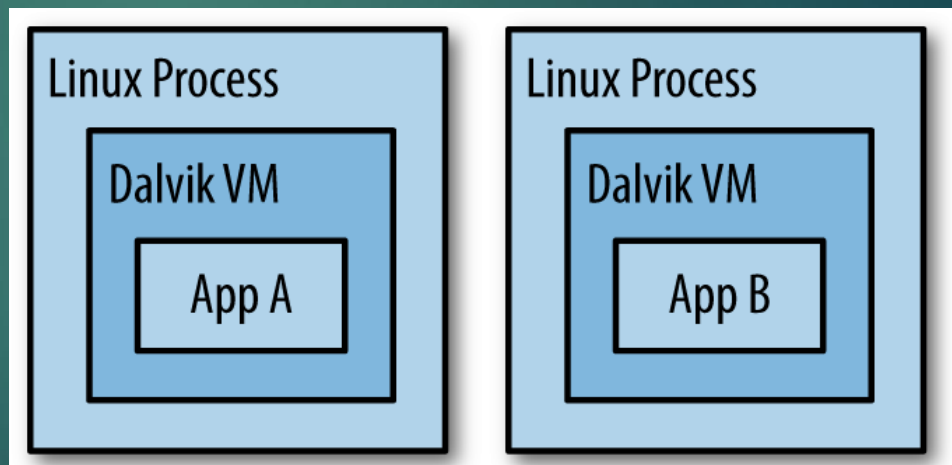
- ▶ Thread
 - ▶ run()
- ▶ Runnable
 - ▶ run()
- ▶ Callable
 - ▶ call()
- ▶ Executor (пул за нишки)
 - ▶ Една
 - ▶ Много
 - ▶ Една

Организация на работата на нишки в Java

- ▶ Приоритети
 - ▶ 1-10
 - ▶ 5
- ▶ `wait()`
- ▶ `notify()`, `notifyAll()`

Организация на процесите в Android

- ▶ Един процес, една нишка - "main" thread
- ▶ Атрибута android:process
 - ▶ <activity>, <service>, <receiver>, <provider>
 - ▶ <application>



Жизнен цикъл на процеса

- ▶ Йерархия на процесите

1. **Foreground process**

2. **Visible process**

- ▶ onPause()

3. **Service process**

- ▶ startService()

4. **Background process**

- ▶ onStop()

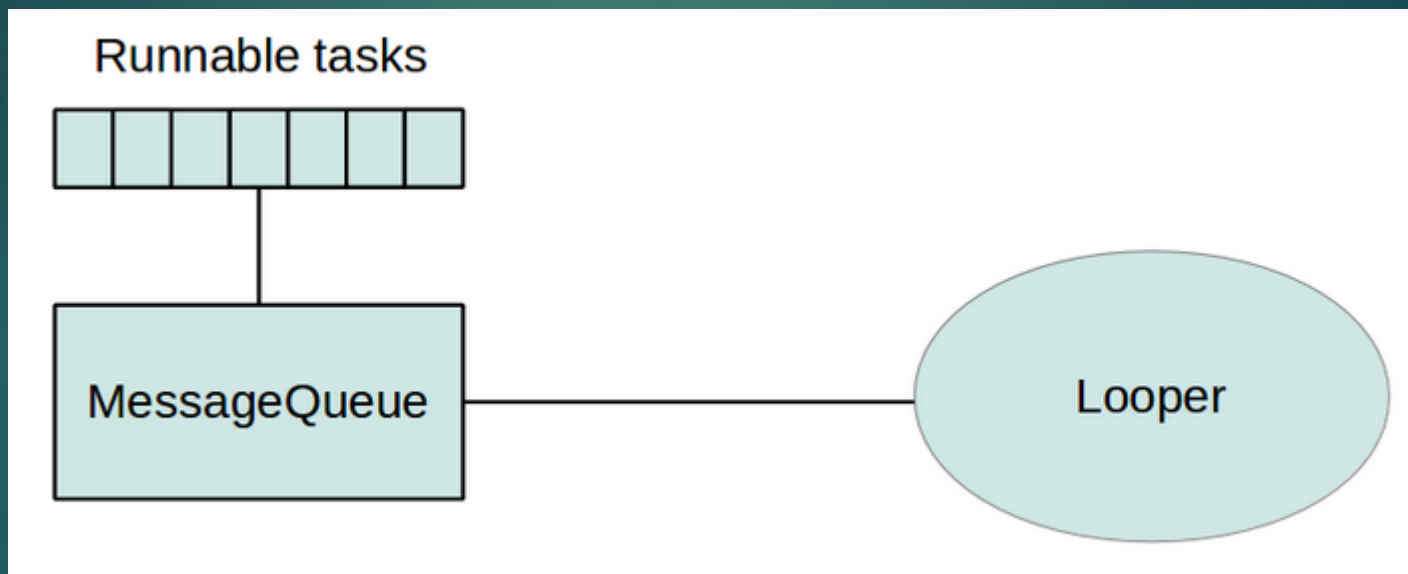
5. **Empty process**

Жизнен цикъл на процеса

- ▶ Определяне на нивото според компонентите
 - ▶ Видим компонент
 - ▶ Междупроцесна зависимост

Организация на процесите. Нишки

- ▶ "main" – диспечер на събития и изгледи



Организация на процесите. Нишки

- ▶ Недостатъци

- ▶ Andoid UI toolkit не е thread-safe

- ▶ "application not responding" (ANR)

1. Не блокирай UI thread

2. Не достъпвай Android UI toolkit извън контекста на UI thread

Работни нишки

- ▶ Worker threads
 - ▶ `Activity.runOnUiThread(Runnable)`
 - ▶ `View.post(Runnable)`
 - ▶ `View.postDelayed(Runnable, long)`

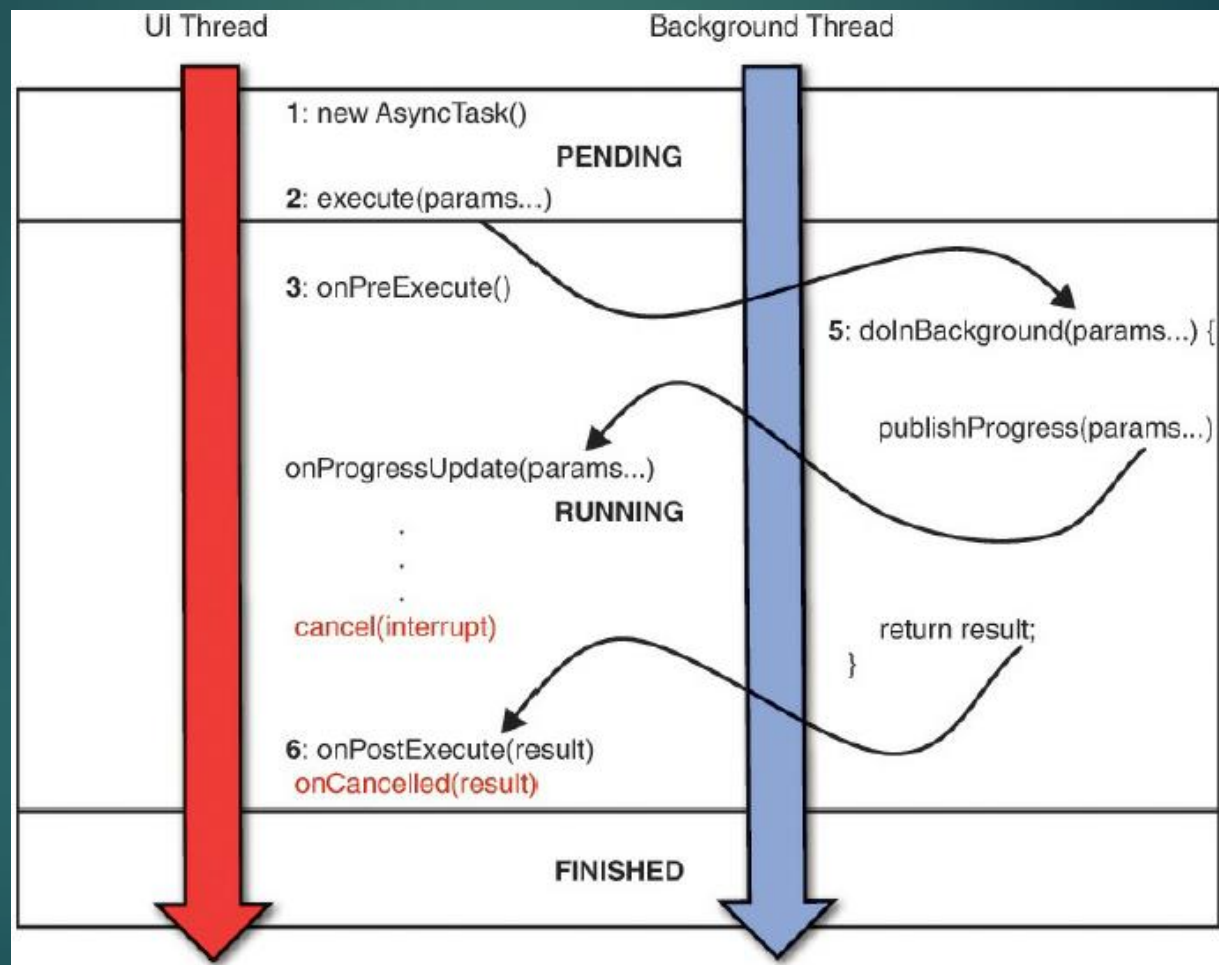
Работни нишки

```
public void onClick(View v) {  
    new Thread(new Runnable() {  
        public void run() {  
            final Bitmap bitmap = processBitMap("image.png");  
            imageView.post(new Runnable() {  
                public void run() {  
                    imageView.setImageBitmap(bitmap);  
                }  
            });  
        }  
    }).start();  
}
```

Класът **AsyncTask**

- ▶ Асинхронна работа на UI интерфейса
 - ▶ Наследяване на `AsyncTask`
 - ▶ Имплементиране на `(doInBackground(...))` и `(onPostExecute())`

Класът AsyncTask



Класът AsyncTask

- ▶ Params - вида на параметрите изпращани на задачата за изпълнение
- ▶ Progress
- ▶ Result - типа на резултата

- ▶ Незадължителност на параметрите
 - ▶ `private class MyTask extends AsyncTask<Void, Void, Void> { ... } - varargs`

- ▶ UI thread
 1. `onPreExecute()`
 2. **`doInBackground(Params...) !!!`**
 - ▶ Нова инстанция
 - ▶ `publishProgress(Progress...)`
 3. `onProgressUpdate(Progress...)`
 4. `onPostExecute(Result)`

Класът AsyncTask

```
private class DownloadFilesTask extends AsyncTask<URL, Integer, Long> {  
    protected Long doInBackground(URL... urls) {  
        int count = urls.length;  
        long totalSize = 0;  
        for (int i = 0; i < count; i++) {  
            totalSize += Downloader.downloadFile(urls[i]);  
            publishProgress((int) ((i / (float) count) * 100));  
  
            if (isCancelled()) break;  
        }  
        return totalSize;  
    }  
  
    protected void onProgressUpdate(Integer... progress) {  
        setProgressPercent(progress[0]);  
    }  
  
    protected void onPostExecute(Long result) {  
        showDialog("Downloaded " + result + " bytes");  
    }  
}
```

Класът `AsyncTask`

- ▶ Стартиране на задача
 - ▶ **new** `DownloadFilesTask().execute(url1, url2, url3)`

Прекратяване на задача

- ▶ `cancel(boolean), isCancelled()`
- ▶ Промени в конфигурацията в реално време

Класът `AsyncTask`

- ▶ Винаги през UI нишката
- ▶ Стартиране само веднъж
- ▶ Гарантира синхронизация между отделните си методи

Проблеми при работа с **AsyncTask**

- ▶ Проблеми с паралелен характер
 - ▶ Достъп до ресурс от UI нишката
- ▶ Проблем при жизнения цикъл
 - ▶ Промяна на конфигурация

Handler

- ▶ Взаимодействие с Looper
- ▶ Принцип на шината
- ▶ Абонамент за съобщения
 - ▶ Опашка за съобщения

Thread-safe МЕТОДИ

- ▶ При необходимост от множествен достъп
 - ▶ bound service

КЛАСИЧЕСКИ АСИНХРОНЕН ПОДХОД

- ▶ принцип на callback методи
 - ▶ механизъм, при който една функция (callback) се предава като аргумент на друга функция или метод, за да бъде извикана по-късно, когато определено събитие настъпи или операция завърши
 - ▶ позволяват **асинхронно програмиране**, при което дадена операция може да продължи, без да блокира текущия поток или нишка

Основни стъпки в работата на callback методите

- ▶ Регистрация на callback метода:
 - ▶ Callback методът се предава като аргумент на друга функция/метод
 - ▶ `performTask(callback);`
- ▶ Изпълнение на задача:
 - Основната функция започва изпълнението на операцията (например бавна мрежова заявка).
- ▶ Извикване на callback:
 - След завършване на операцията или при настъпване на събитие, callback методът се извиква.
 - Callback-ът получава резултата от операцията (ако има такъв) като аргумент.

Пример за работа на callback метод в Java

```
public interface Callback {  
    void onSuccess(String result);  
    void onError(Exception e);  
}
```

Пример за работа на callback метод в Java

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class TaskRunner {

    private final ExecutorService executor =
        Executors.newSingleThreadExecutor();

    public void performTask(Callback callback) {
        executor.submit(() -> {
            try {
                // Симулиране на бавна задача
                Thread.sleep(2000);

                String result = "Task completed successfully";

                // Уведомяване за успех
                callback.onSuccess(result);
            } catch (Exception e) {
                // Уведомяване за грешка
                callback.onError(e);
            }
        });
    }
}
```

Пример за работа на callback метод в Java

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Task started...");  
        TaskRunner runner = new TaskRunner();  
    }  
}  
  
runner.performTask(new Callback() {  
    @Override  
    public void onSuccess(String result) {  
        System.out.println("Success: " + result);  
    }  
  
    @Override  
    public void onError(Exception e) {  
        System.err.println("Error: " + e.getMessage());  
    }  
});
```

Пример за работа на callback метод в Java

1. Регистрация:

- ▶ Callback се предава като аргумент на метода `performTask`.
- ▶ В този пример, предоставеният callback има две дефинирани операции: `onSuccess` и `onError`.

2. Изпълнение:

- ▶ Методът `performTask` изпълнява бавна задача във фонов поток с помощта на `ExecutorService`.

3. Уведомяване:

- ▶ Ако задачата приключи успешно, `callback.onSuccess(result)` се извиква.
- ▶ Ако възникне грешка, `callback.onError(e)` се извиква.

4. Реакция:

- ▶ В `main` функцията, callback-ът обработва резултата или грешката и извежда съответното съобщение.

Силни страни на callback методите

▶ Асинхронност:

- Позволяват изпълнение на бавни задачи, без да блокират главния поток или нишка.

▶ Гъвкавост:

- Позволяват изпълнение на различни действия в зависимост от резултата на операцията.

▶ Интеграция:

- Callback методите са широко използвани в мрежови операции, анимации, бази данни и много други контексти.

Ограничения на callback методите

- ▶ Callback Hell:
 - ▶ При сложни задачи с множество вложени callback-и кодът може да стане труден за четене и поддръжка.
 - ▶ Трудност при управление на грешки:
 - ▶ Грешките трябва да се управляват ръчно във всеки callback.
- ▶ Непоследователен код:
 - ▶ Callback методите могат да разделят изпълнението на логиката на множество места, което прави кода по-труден за следене.

Какво представлява "Callback Hell"?

- ▶ когато имаме много вложени callback функции, което води до труден за четене и поддръжка код

```
performTask(result -> {  
  anotherTask(result, nextResult -> {  
    finalTask(nextResult, finalResult -> {  
      System.out.println("Final result: " + finalResult);  
    }, error -> {  
      System.err.println("Error in final task");  
    });  
  }, error -> {  
    System.err.println("Error in another task");  
  });  
}, error -> {  
  System.err.println("Error in performTask");  
});
```


Алтернативи на callback МЕТОДИТЕ

► Promises/Futures:

- Позволяват управление на асинхронни задачи чрез "обещания" за бъдещ резултат.
- Пример: `CompletableFuture` в Java.

► Kotlin Coroutines:

- Използват `suspend` функции, което прави кода по-четивен и лесен за поддръжка.

► RxJava:

- Осигурява реактивно програмиране с потоци от данни.

Kotlin coroutines

- ▶ леки изпълнителни единици, използвани за асинхронно програмиране
- ▶ могат да бъдат суспендирани и възобновени, без да блокират текущата нишка
- ▶ интегрирани в Kotlin

Как работят корутините?

► Лекота:

- Корутините не създават нова нишка. Вместо това те използват съществуващи нишки и могат да споделят една нишка с множество корутини.

► Блокиране и възобновяване:

- Корутината може да бъде временно "замразена" (suspend), докато чака резултат, и след това "събудена" и продължена от същата точка, когато резултатът е готов.

► Асинхронност:

- Позволяват писане на асинхронен код по **синхронен начин**, което го прави по-четим и лесен за поддръжка.

Ключови елементи на корутините

► **suspend** функции:

- Функции, които могат да бъдат блокиране и възобновени. Използват се само в корутинен контекст.
- ```
suspend fun fetchData(): String {
 delay(1000L) // Суспендира корутината за 1 секунда
 return "Data fetched"
}
```

# Ключови елементи на корутините

## ▶ Корутинен билдър:

- Стартира нова корутина.
- Основни билдъри в Kotlin:
  - **launch**: Стартира корутина, която не връща резултат.
  - **async**: Стартира корутина, която връща резултат чрез Deferred.

## ▶ CoroutineScope:

- Определя обхвата на корутината и контролира нейния жизнен цикъл.
- Примери: GlobalScope, runBlocking.

## ▶ Диспечер (Dispatcher):

- Контролира на коя нишка или контекст ще се изпълни корутината.
  - **Dispatchers.Main**: Главната (UI) нишка.
  - **Dispatchers.IO**: За операции с вход/изход (мрежови заявки, четене/запис в база данни).
  - **Dispatchers.Default**: За тежки изчисления.

# Пример за корутина в Kotlin

```
import kotlinx.coroutines.*

fun main() = runBlocking {
 println("Start: ${Thread.currentThread().name}")

 // Стартира корутина
 val job = launch {
 println("Running in: ${Thread.currentThread().name}")
 delay(2000L) // Суспендира корутината за 2 секунди
 println("Completed in: ${Thread.currentThread().name}")
 }

 println("Waiting...")
 job.join() // Изчаква корутината да завърши

 println("End: ${Thread.currentThread().name}")
}
```

# Пример за корутина в Kotlin

## ▶ **runBlocking:**

- Създава корутинен обхват, който блокира текущата нишка (в случая `main`), докато всички корутини в него завършат.

## ▶ **launch:**

- Стартира нова корутина за изпълнение на асинхронна операция.

## ▶ **delay:**

- Блокира корутината за даден период от време, без да блокира нишката.

## ▶ **job.join():**

- Изчаква текущата корутина да завърши.

# Пример за паралелно изпълнение с `async`

```
import kotlinx.coroutines.*

fun main() = runBlocking {
 val timeStart = System.currentTimeMillis()

 val task1 = async { performTask(1, 2000L) }
 val task2 = async { performTask(2, 3000L) }

 println("Result 1: ${task1.await()}")
 println("Result 2: ${task2.await()}")

 val timeEnd = System.currentTimeMillis()
 println("Total time: ${timeEnd - timeStart} ms")
}

suspend fun performTask(taskId: Int, delayTime: Long): String {
 delay(delayTime)
 return "Task $taskId completed"
}
```



# Пример за паралелно изпълнение с `async`

## 1. `async`:

- ▶ Стартира паралелно изпълнение на две корутини (`task1` и `task2`).
- ▶ Връща обект от тип `Deferred`, който може да се използва за извикване на `await()` и получаване на резултата.

## 2. `await()`:

- ▶ Изчаква резултата от асинхронната операция.

# Предимства на корутините

- ▶ **Асинхронно програмиране без сложност:**
  - Кодът изглежда линейно, въпреки че е асинхронен.
- ▶ **Не блокират нишките:**
  - Корутините суспендират своето изпълнение, без да блокират текущата нишка.
- ▶ **Мащабируемост:**
  - Една нишка може да управлява хиляди корутини.
- ▶ **Гъвкавост:**
  - Позволяват лесно превключване между контексти (например от `Dispatchers.IO` към `Dispatchers.Main`).

# Недостатъци на корутините

## ► Изискват коректен жизнен цикъл:

- Ако корутината не бъде отменена правилно, тя може да причини течове на памет (memory leaks).

## ► Не са магически:

- Лошо проектирани корутини или прекомерно създаване на корутини могат да доведат до проблеми с производителността.

## ► Обучение:

- Разработчиците трябва да научат основите на корутините и как да ги използват правилно.

# Кога да използваме корутини?

## ▶ Асинхронни задачи:

- Мрежови операции, достъп до база данни, работа с файлове.

## ▶ Паралелни операции:

- Изчисления, които могат да се изпълняват паралелно.

## ▶ Анимации и UI операции:

- Работа с анимации или обновяване на UI в Android.